



SCAD COLLEGE OF ENGINEERING & TECHNOLOGY

Recognized under Section 2(f) & 12(B) of the UGC Act, 1956

Approved by AICTE Affiliated to Anna University, Chennai

Cheranmahadevi, Tirunelveli - 627414

CCS333 – SOFTWARE DEFINED NETWORK LABORATORY

LAB RECORD

NAME:

REGISTER NUMBER:

DEGREE&BRANCH:

YEAR/SEMESTER:



Certified that this is a Bonafide record work done

By Selvan/Selvi.....

with Register Nostudying in III-year VI semester in Computer Science and Engineering branch of this Institution during the academic year 2024-2025 [EVEN SEM]

Staff in-charge

Head of the Department

Submitted for the Anna University practical examination held at SCAD College of Engineering and Technology, Cherranmahadevi on

Internal Examiner

External Examiner

LIST OF EXPERIMENTS

Sl.No.	Date	Experiment Name	Mark	Signature
1a		Setup your own virtual SDN lab Virtualbox / Mininet Environment for SDN		
1b		Setting up own virtual SDN lab using kathara		
1c		Setting virtual SDN lab Installation of GNS3		
2		Create a simple mininet topology with SDN controller and use Wireshark to capture and visualize the OpenFlow messages such as OpenFlow FLOW MOD, PACKET IN, PACKET OUT etc.		
3		Create a SDN application that uses the Northbound API to program flow table rules on the switch for various use cases like L2 learning switch, Traffic Engineering, Firewall etc.		
4		Create a simple end-to-end network service with two VNFs using vim-emu https://github.com/containernet/vim-emu		
5		Install OSM and onboard and orchestrate network service		

Ex. No: 1a DATE:	Setting virtual SDN lab Installation of Virtual box and Mininet Environment for SDN
-----------------------------------	---

AIM:

To Setup virtual SDN lab in Mininet Environment for SDN.

ALGORITHM:

Step 1: Download Virtual Box from the official website <https://www.virtualbox.org/>.

Step 2: Install the downloaded latest version(7.0.14) of Virtual Box for windows OS.

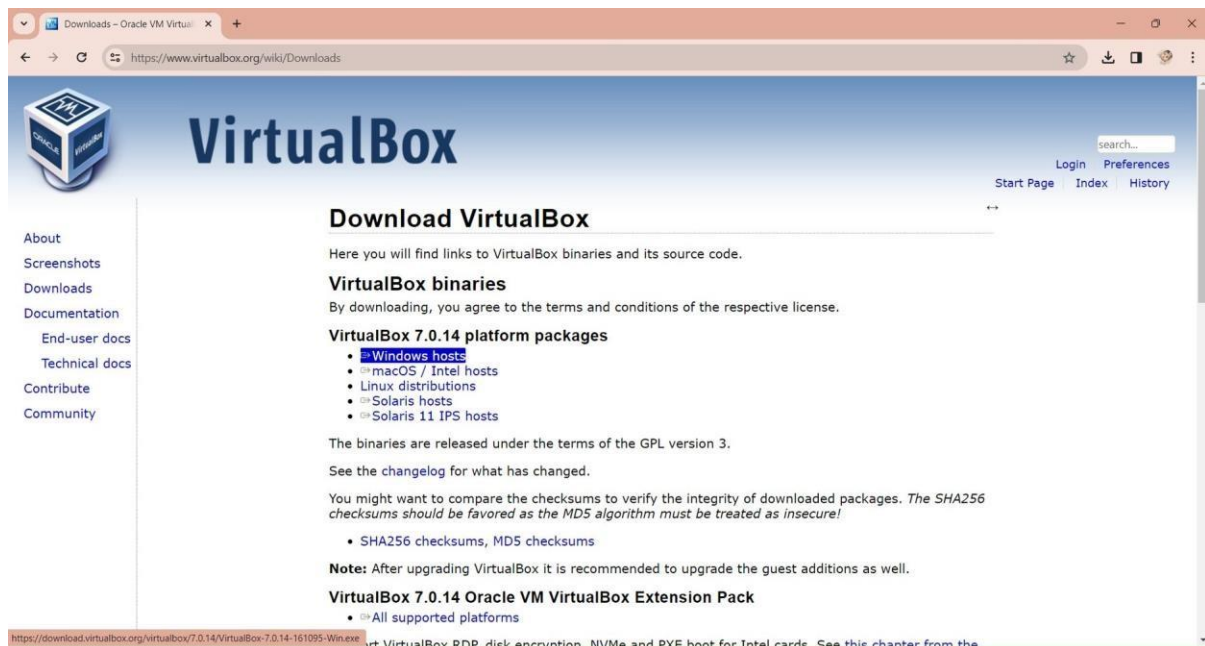
Step 3: Steps for downloading Mininet :

1. To Install Mininet visit the official website <https://mininet.org/download/>.
2. Then Click the option **Mininet Releases** to download Mininet Image.
3. And then download latest version(2.3.0) of Mininet from Assets.
4. Import the downloaded Mininet from the virtualbox.

PROCEDURE:

Step 1:

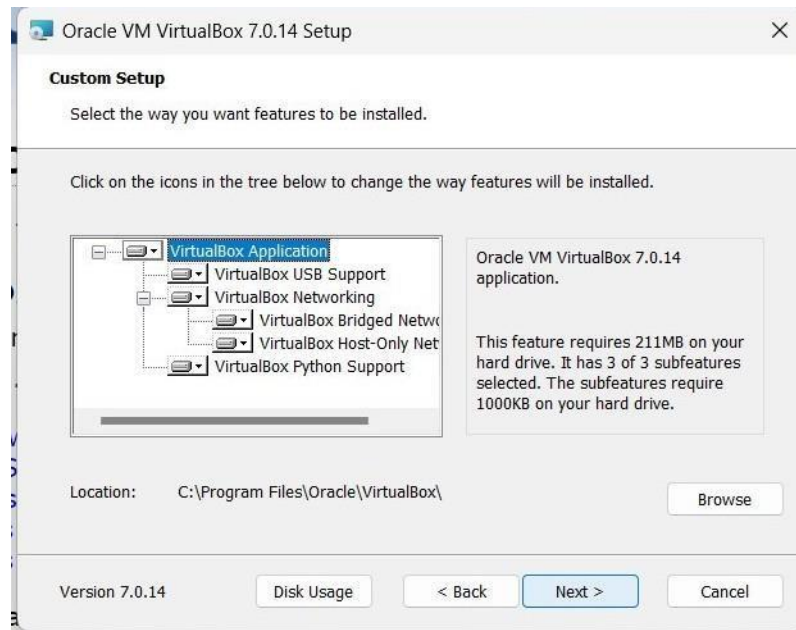
Download Virtual Box from the official website <https://www.virtualbox.org/wiki/Downloads>



Step 2:

Install the downloaded latest version(7.0.14) of Virtual Box for windows OS by clicking the Downloaded VirtualBox application.

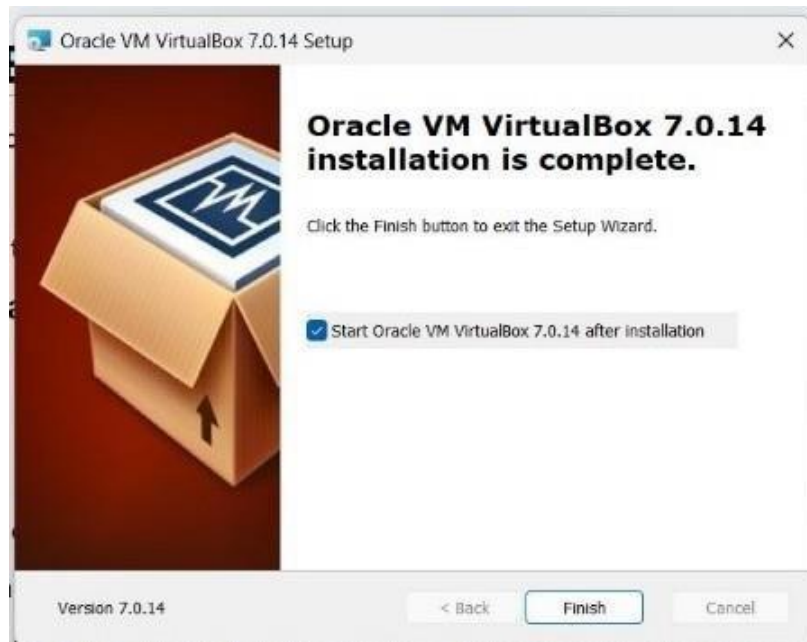
It will show the Setup wizard as shown below click the next button.



Click the Yes button.



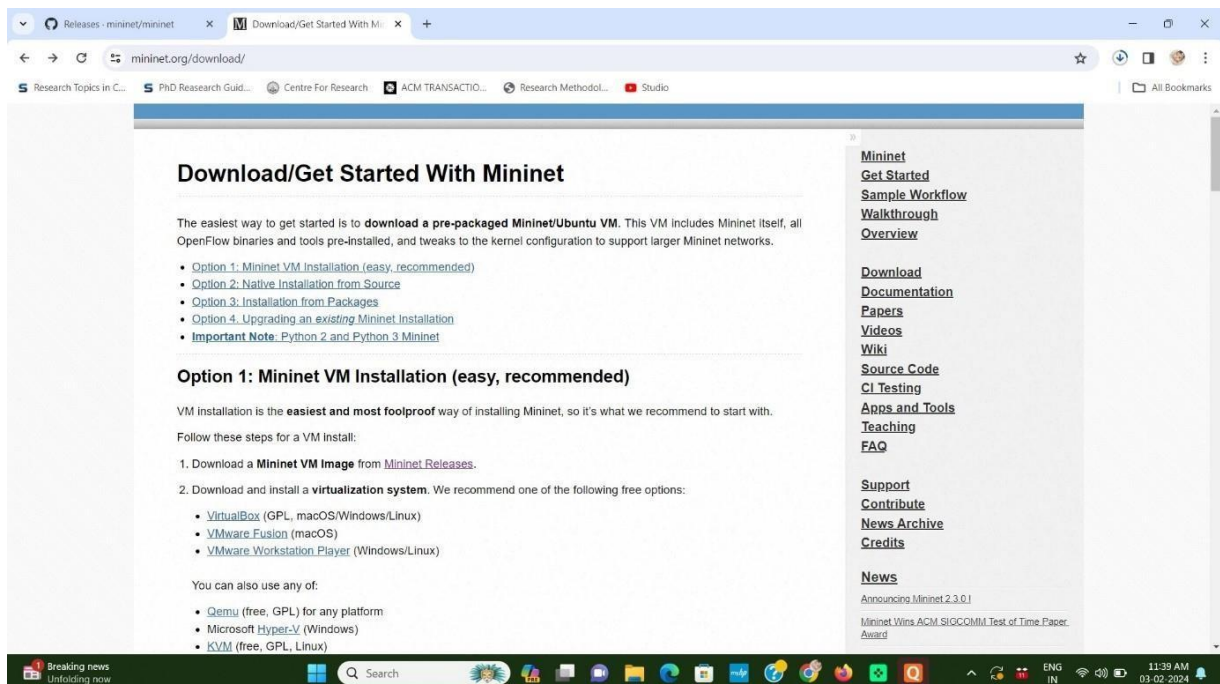
Click the finish button to finish the installation.



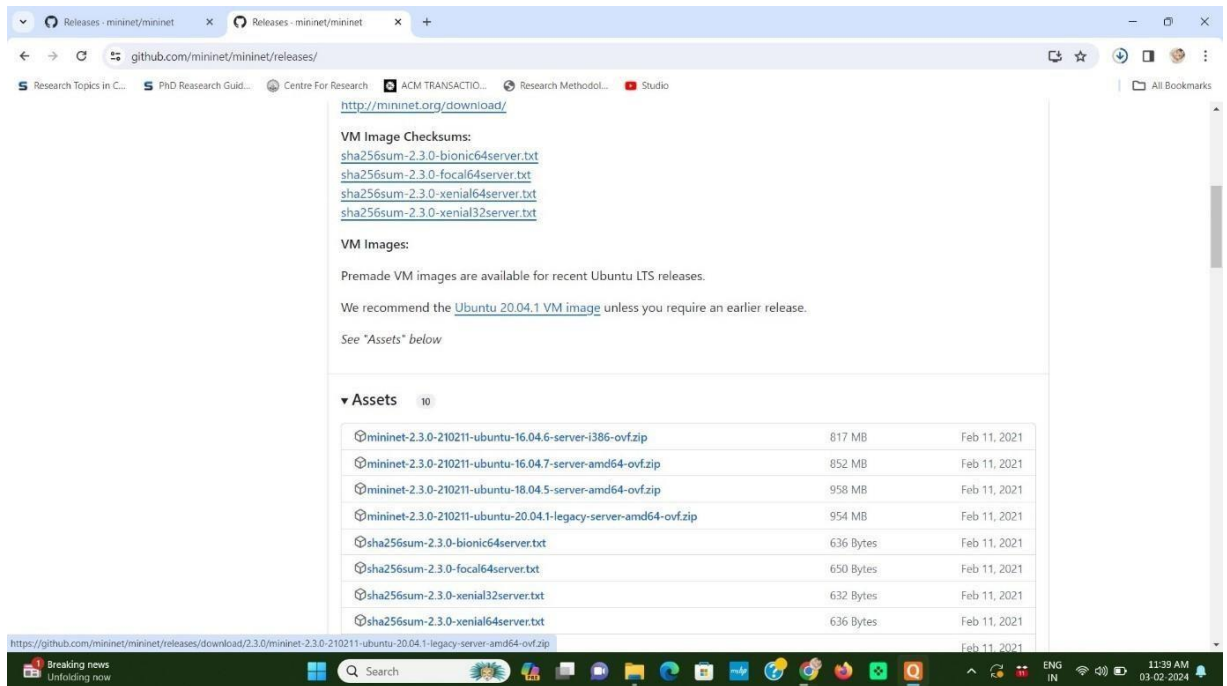
Step 3:

The following steps are to download Mininet.

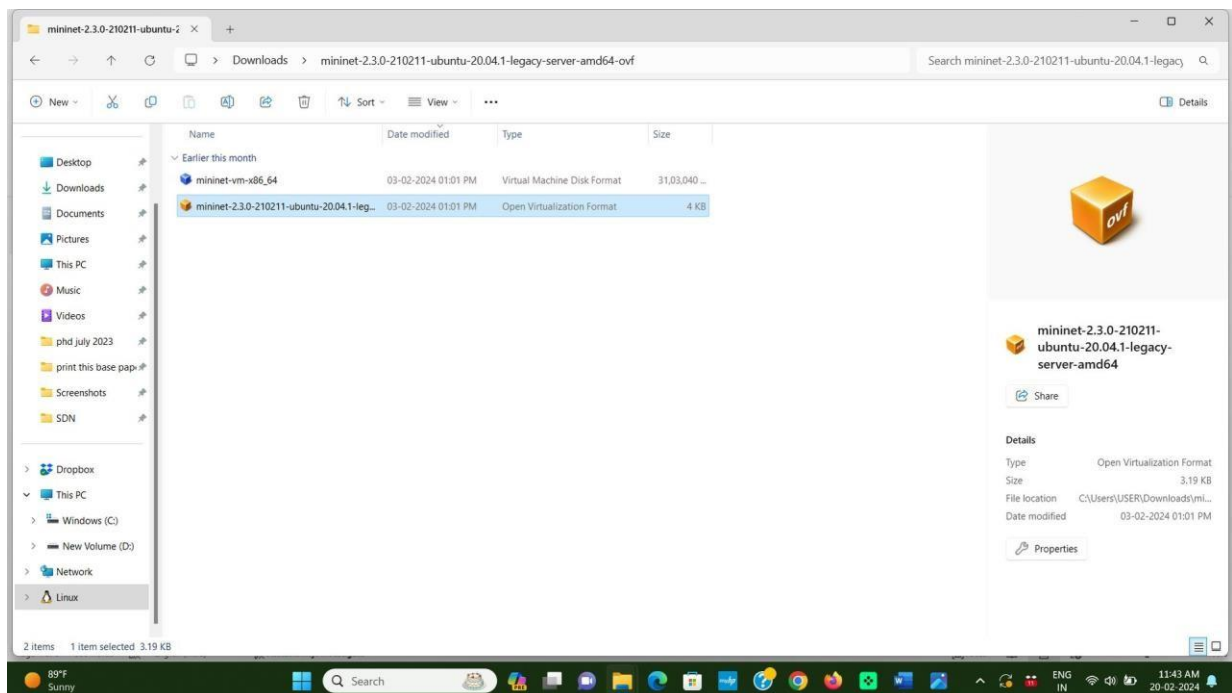
1. To Install Mininet visit the official website <https://mininet.org/download/>.
2. Then Click the option **Mininet Releases** to download Mininet VM Image.



3. And then download latest version(2.3.0) of Mininet from Assets.



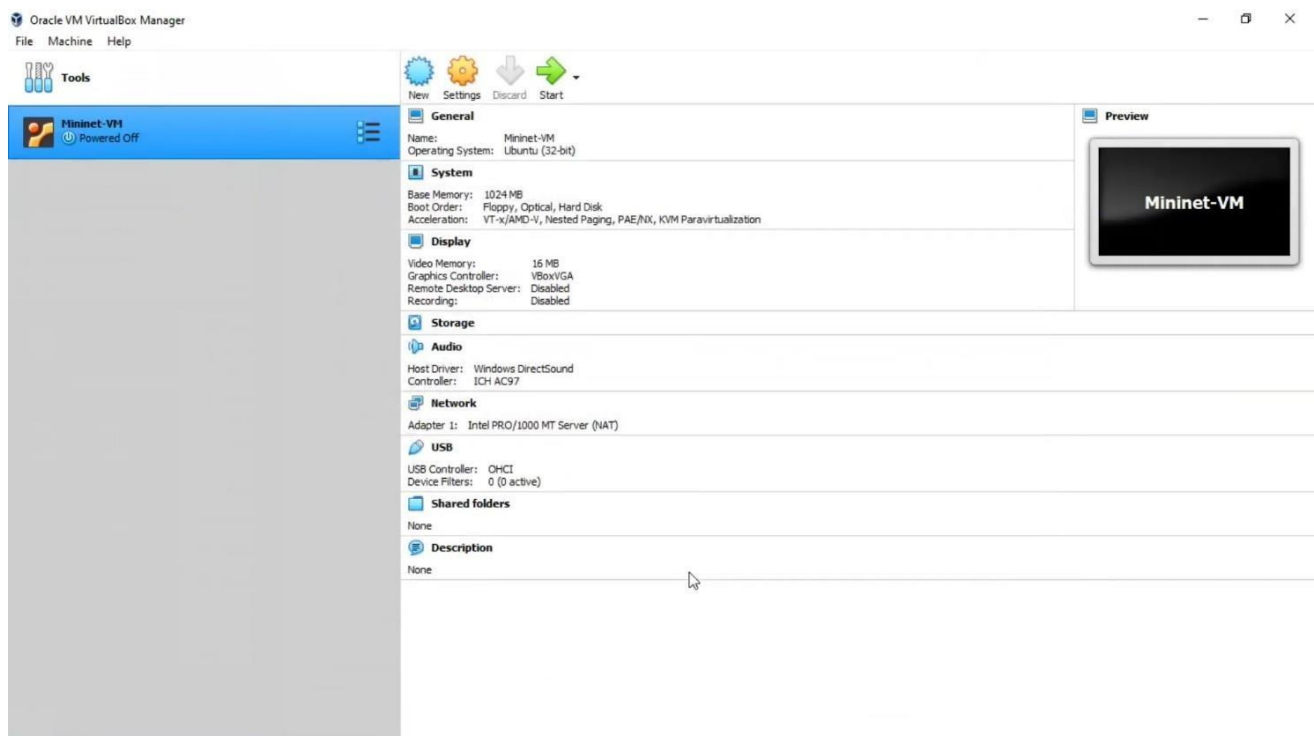
4. Import the downloaded Mininet from the virtualbox.
 - a) Extract the Downloaded zipper file.



- b) From the mininet folder double click the “mininet-2.3.0-210211-ubuntu-20.04.1-legacy-server-amd64.ovf” file



c) Mininet imported successfully in the Virtual Box.



Result:

Thus the installation of Virtual box and Mininet Environment for SDN was done successfully.

Ex. no:1b DATE:	Setting up own virtual SDN lab using kathara
--------------------	--

AIM:

To setup virtual SDN lab using Kathara.

ALGORITHM:

Step 1: Download Kathara from the official website <https://www.kathara.org/>

Step 2: Click the option Download for Windows.

Step 3: Open the downloaded kathara and complete the installation wizard.

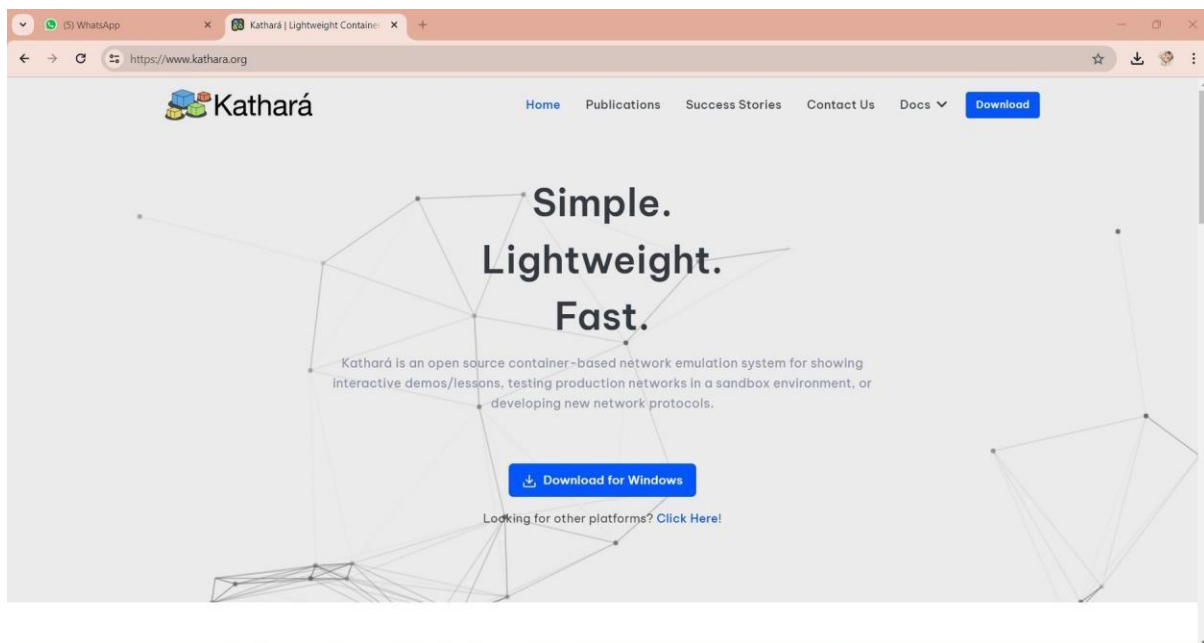
Step 4: Click install option to install kathara.

Step 5: Click Finish option after completing all the installation process.

PROCEDURE:

Step 1:

Download Kathara from the Official Website <https://www.kathara.org/wiki/Download>

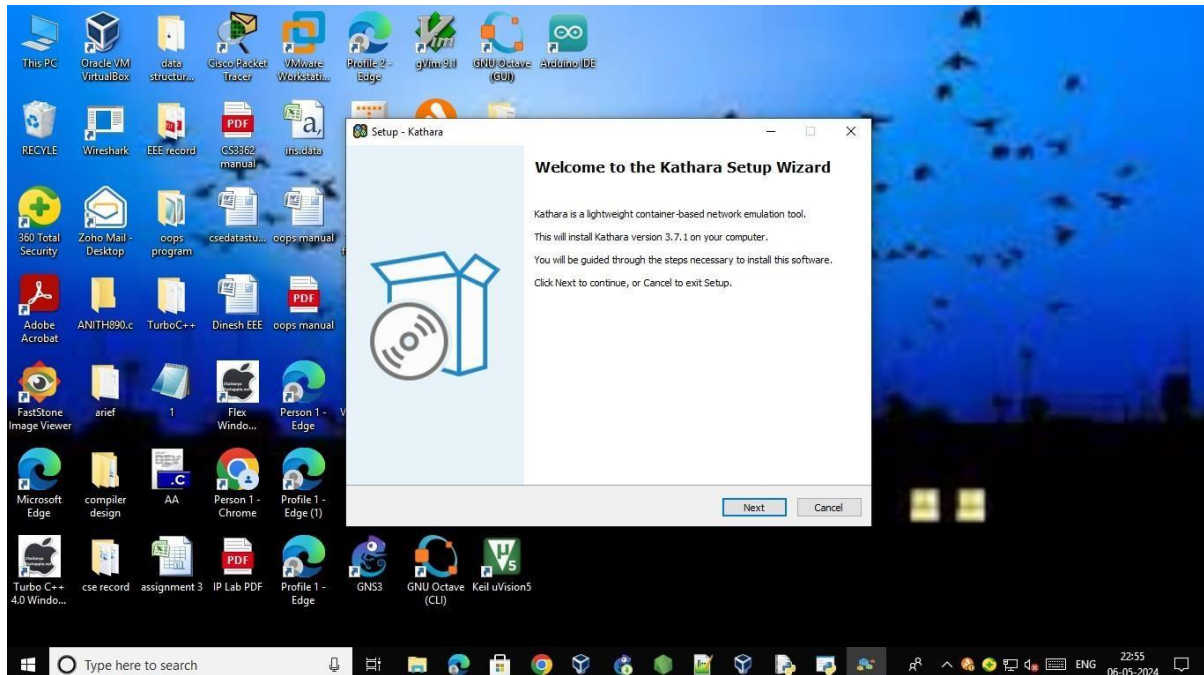


Step 2:

Click the option Download for windows.

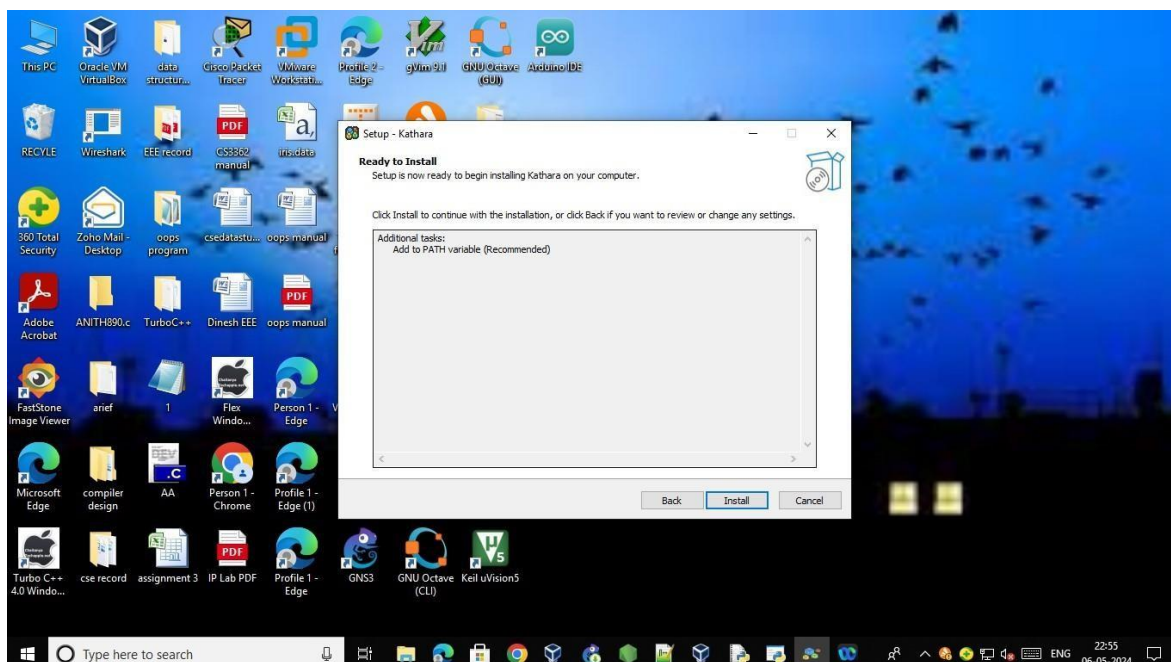
Step 3:

Open the downloaded kathara and complete the installation wizard.



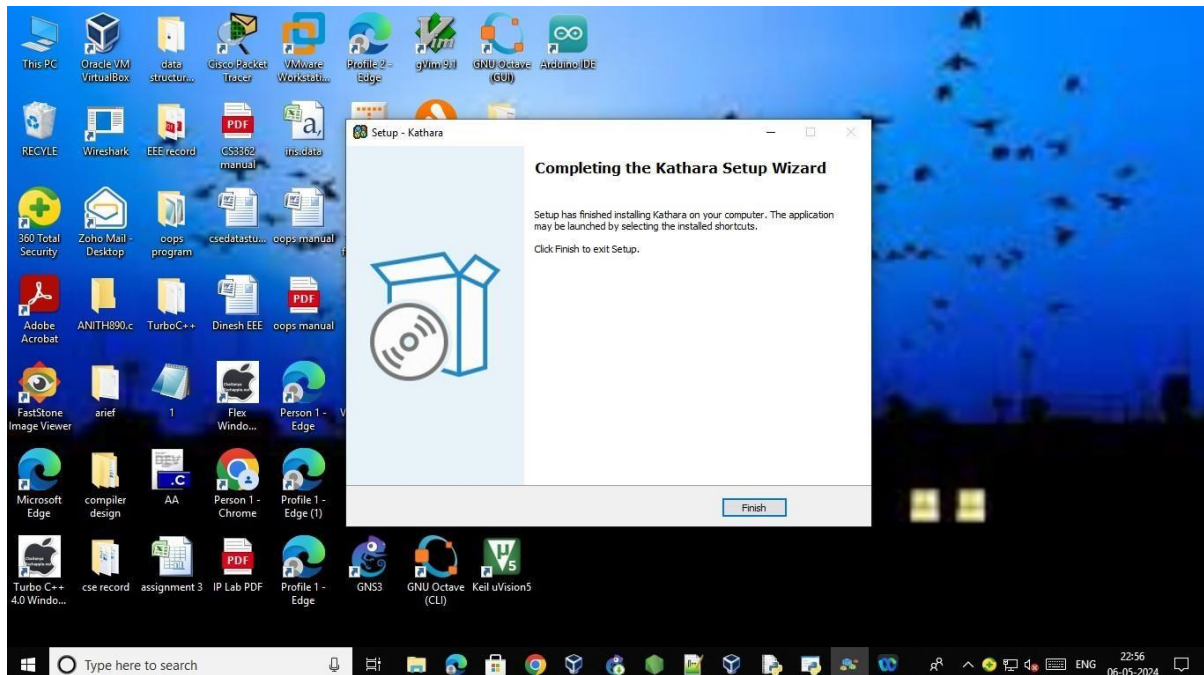
Step 4:

Click install option to install kathara.



Step 5:

Click Finish option after completing all the installation process.



Result:

Thus the installation of Kathara has been successfully completed.

Ex.No:1c
DATE:

Setting virtual SDN lab

Installation of GNS3

AIM:

To install the GNS3 to setup SDN lab.

ALGORITHM:

Step1: Download the GNS3 from website <https://www.gns3.com/software/download-vm> and extract the zip file.

Step2: Open VirtualBox and select import.

Step 3: Select the location of the extracted GNS3 file.

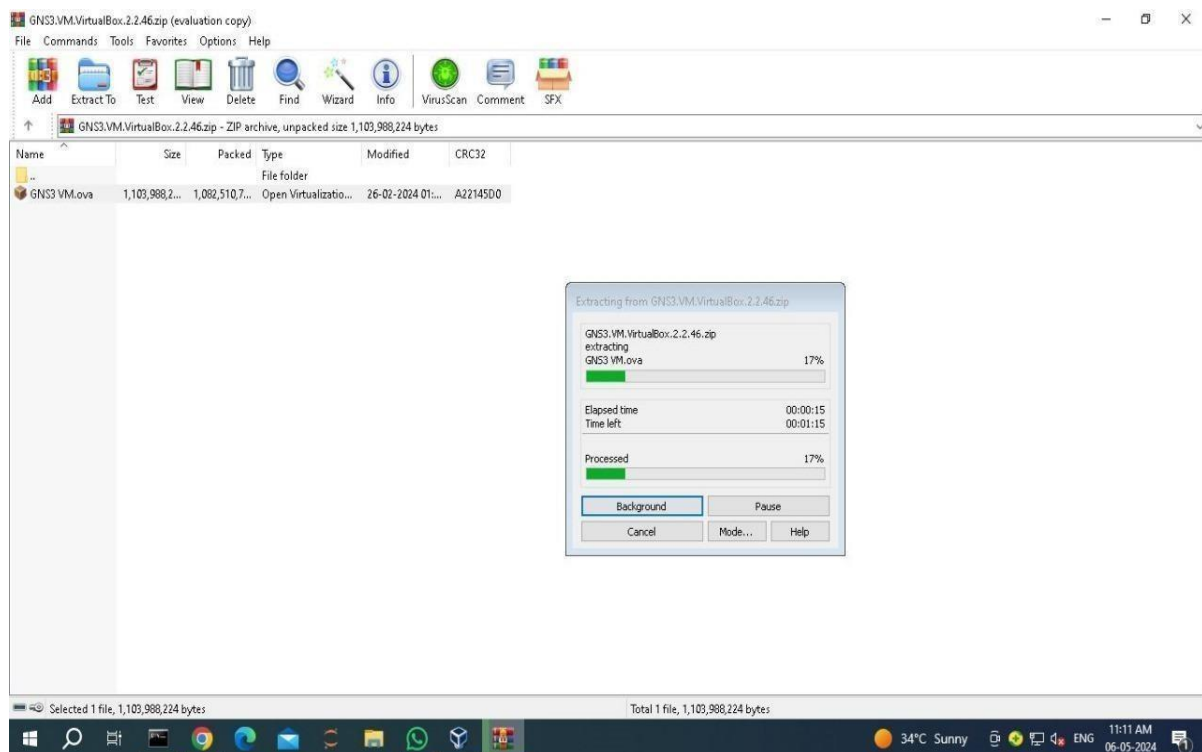
Step 4: Click the finish button to finish the installation.

Step 5: Select the GNS3 vm and start.

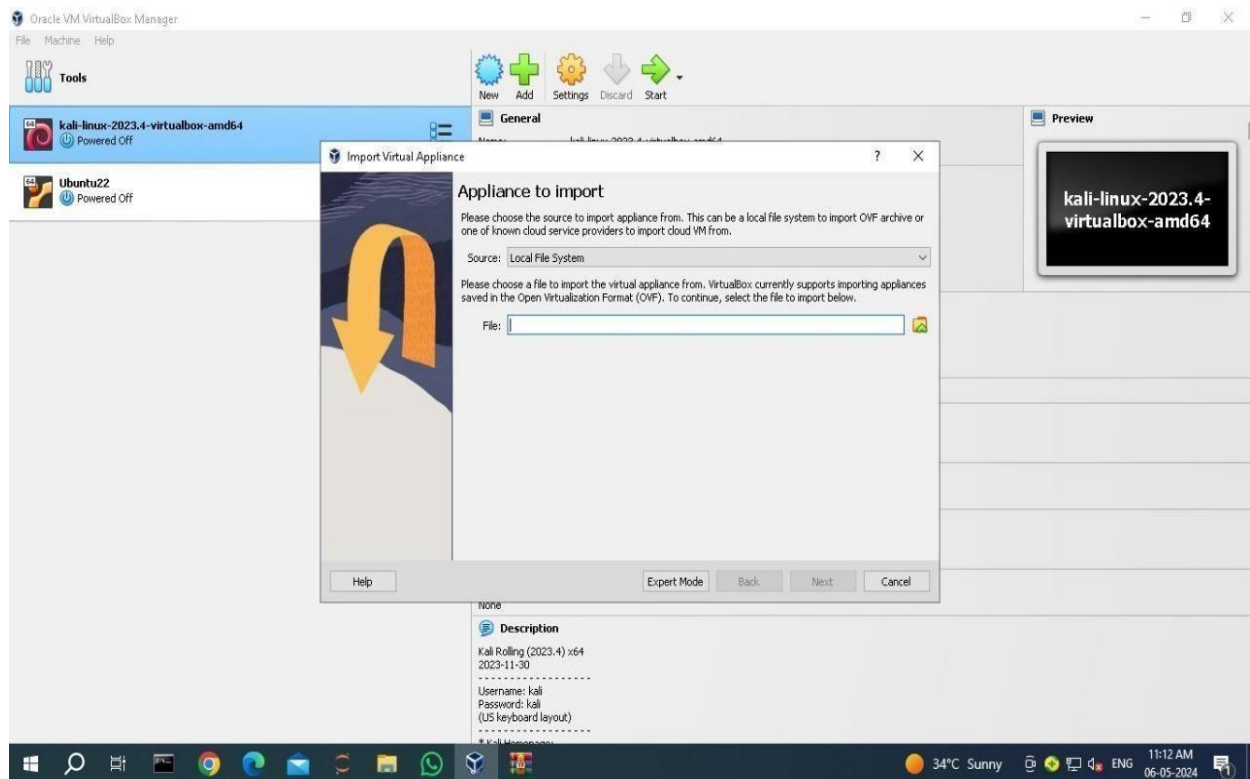
PROCEDURE:

Step1:

Download the GNS3 from website <https://www.gns3.com/software/download-vm> and extract the zip file.

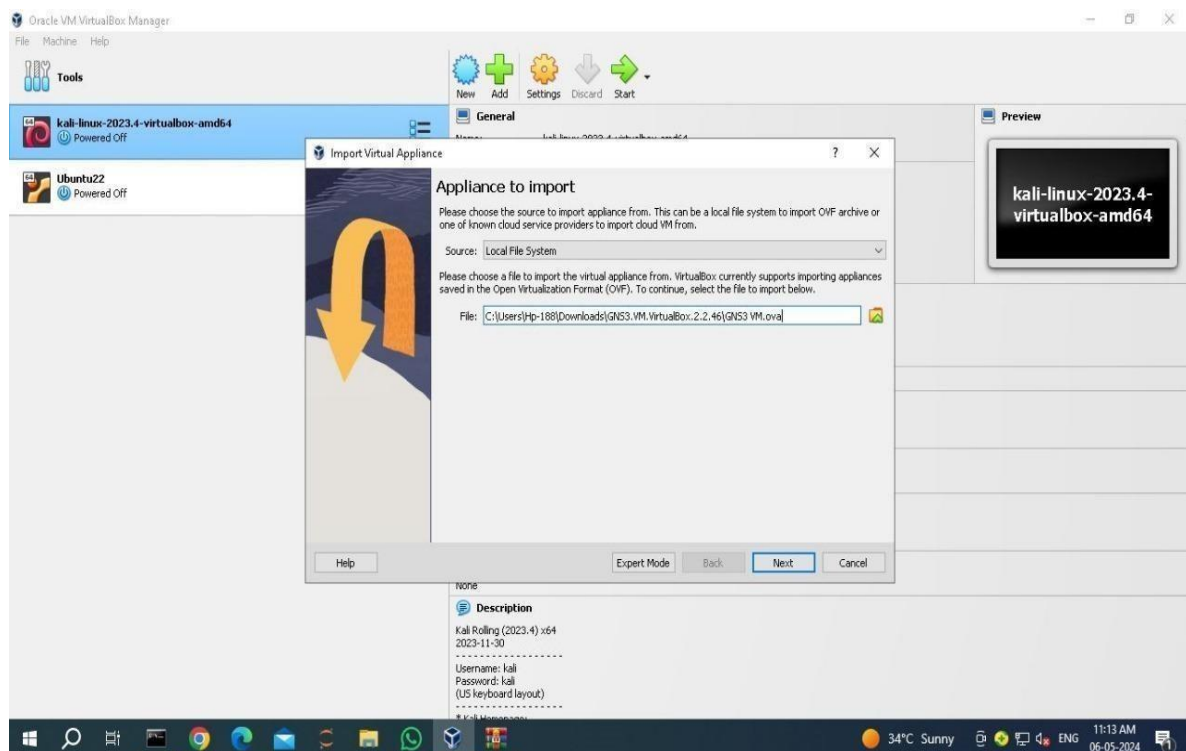


Step2:OpenVirtualBoxandselectimport.

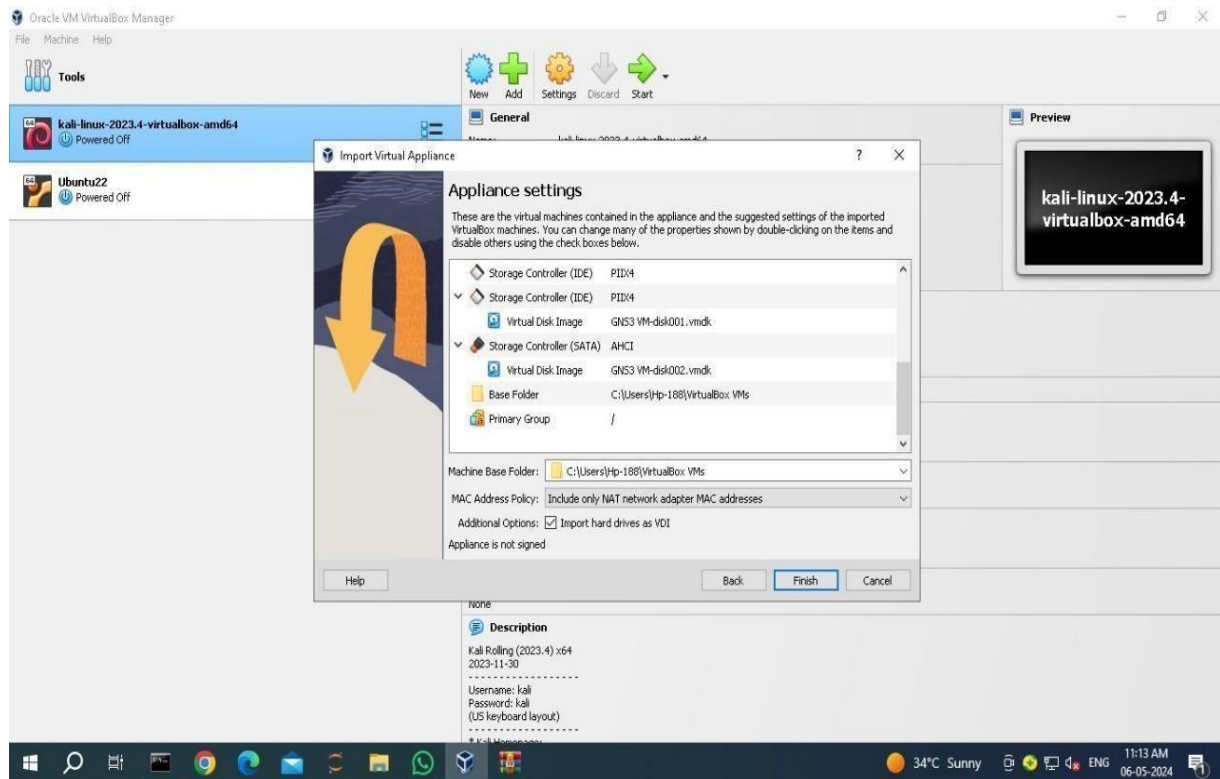


Step 3:

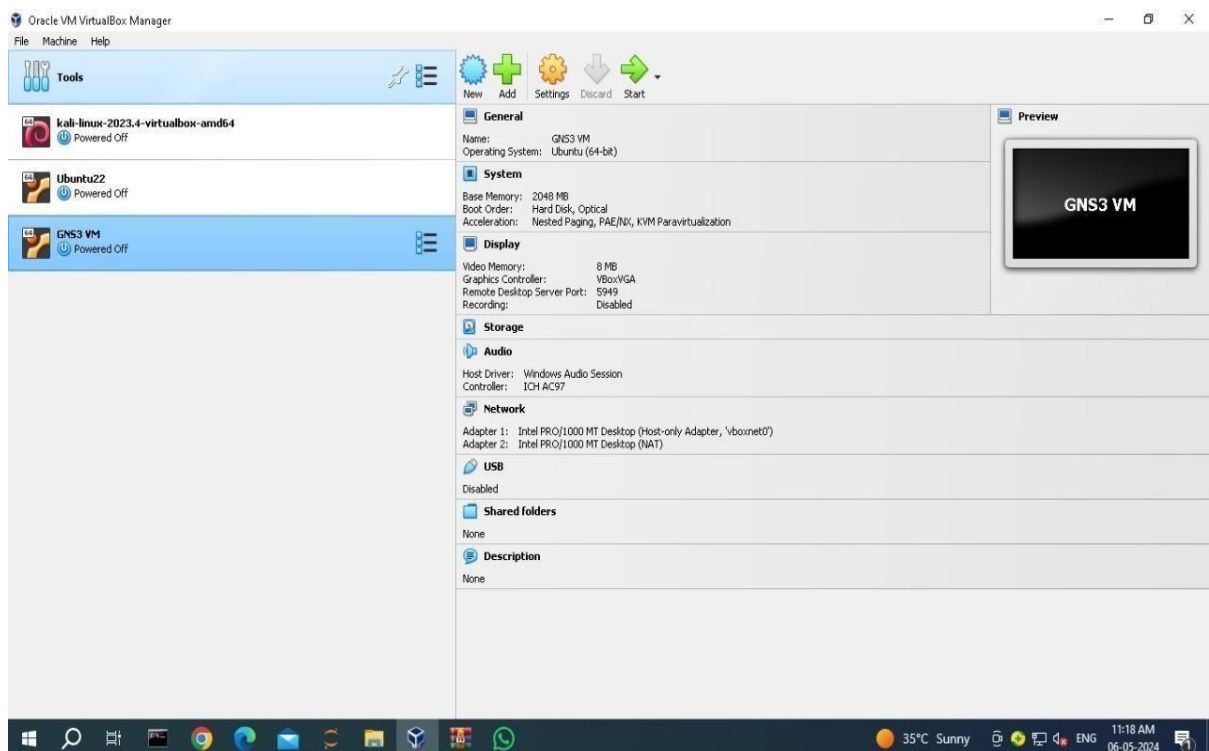
Select the location of the extracted GNS3 file.

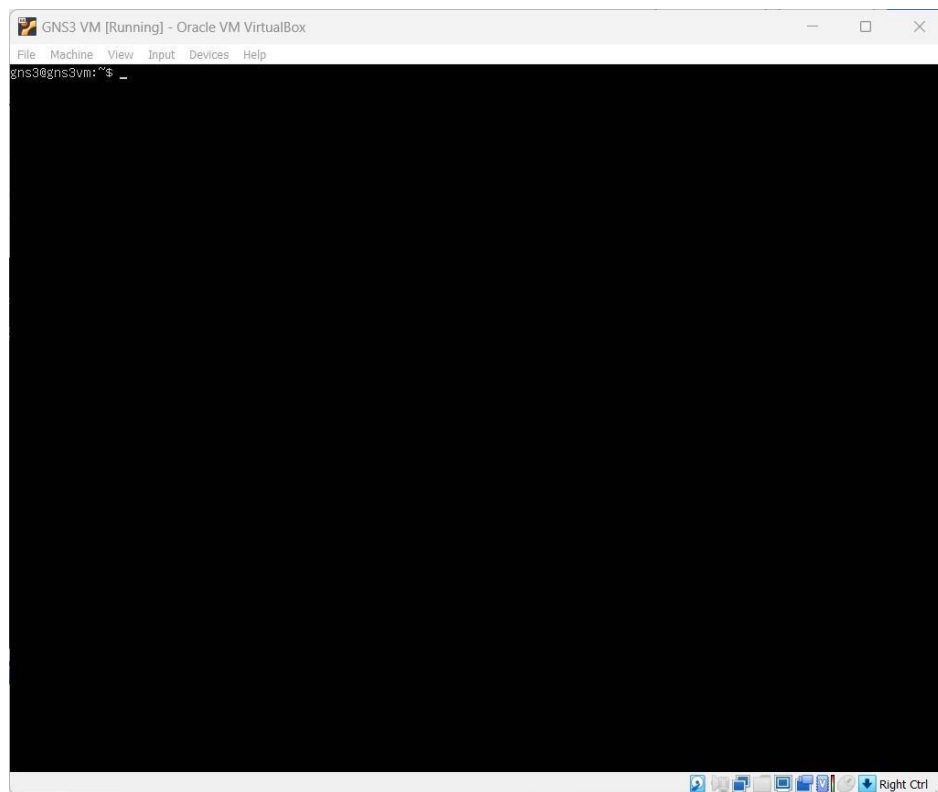
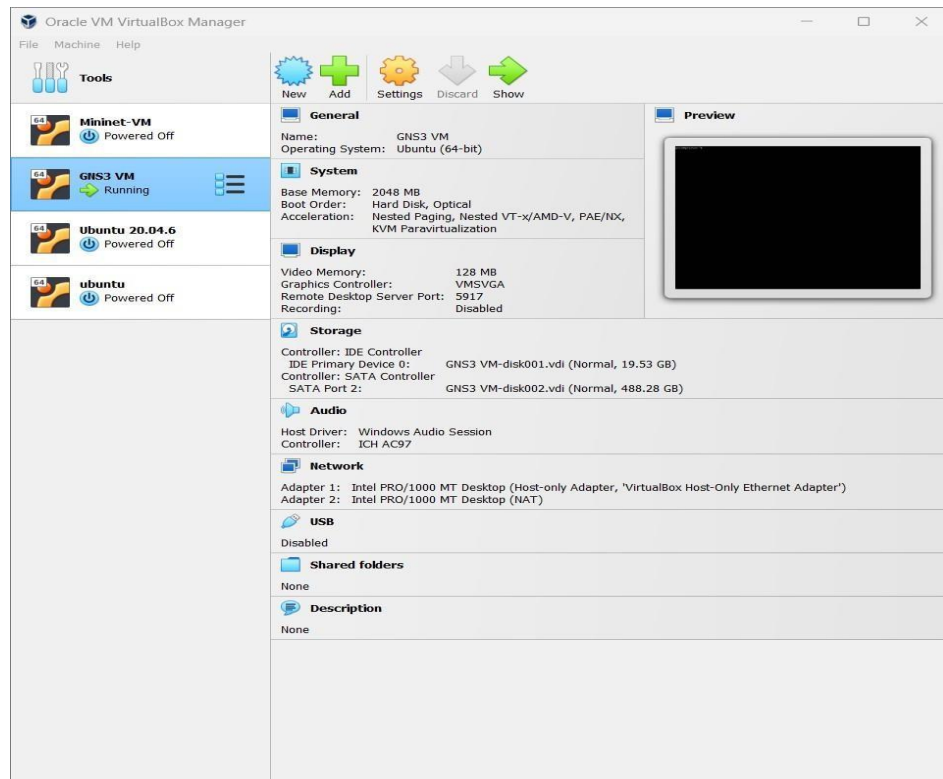


Step3:Clickthefinishbuttontofinishtheinstallation.



Step4:SelecttheGNS3vmmandStart.





Result:

Thus the GNS3 was installed successfully.

Ex. No: 2
DATE:

Create a simple mininet topology with SDN controller and use Wireshark to capture and visualize the OpenFlow messages such as OpenFlow FLOW MOD, PACKET IN, PACKET OUT etc.

AIM:

To create a simple mininet topology with SDN controller and to use Wireshark to capture and visualize the OpenFlow messages such as OpenFlow FLOW MOD, PACKET IN, PACKET OUT.

ALGORITHM:

Step 1: Install python3.

Step 2: Install the ryu controller.

Step 3: Install the mininet in ubuntu.

Step 4: Create the mininet network.

Step 5: Create a simple mininet topology.

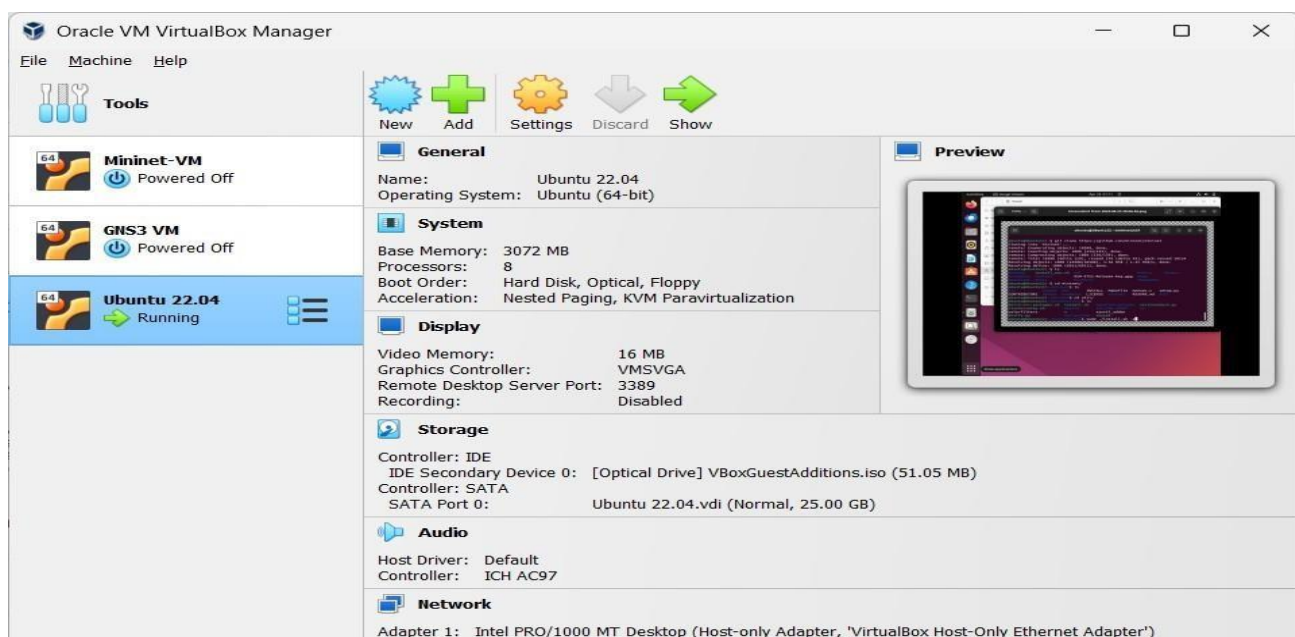
Step 6: Configure hosts in network.

Step 7: Use Wireshark to capture and visualize OpenFlow messages.

PROCEDURE:

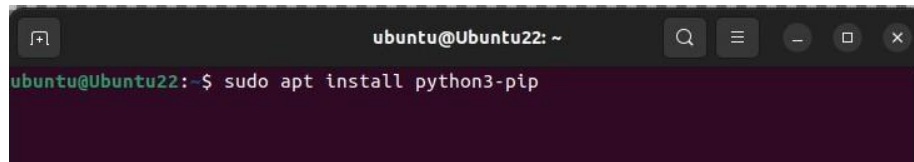
Step 1:

Open the Oracle Virtual box, and then start the Ubuntu 22.04.



Install python3 using the following command.

```
sudo apt install python3-pip
```

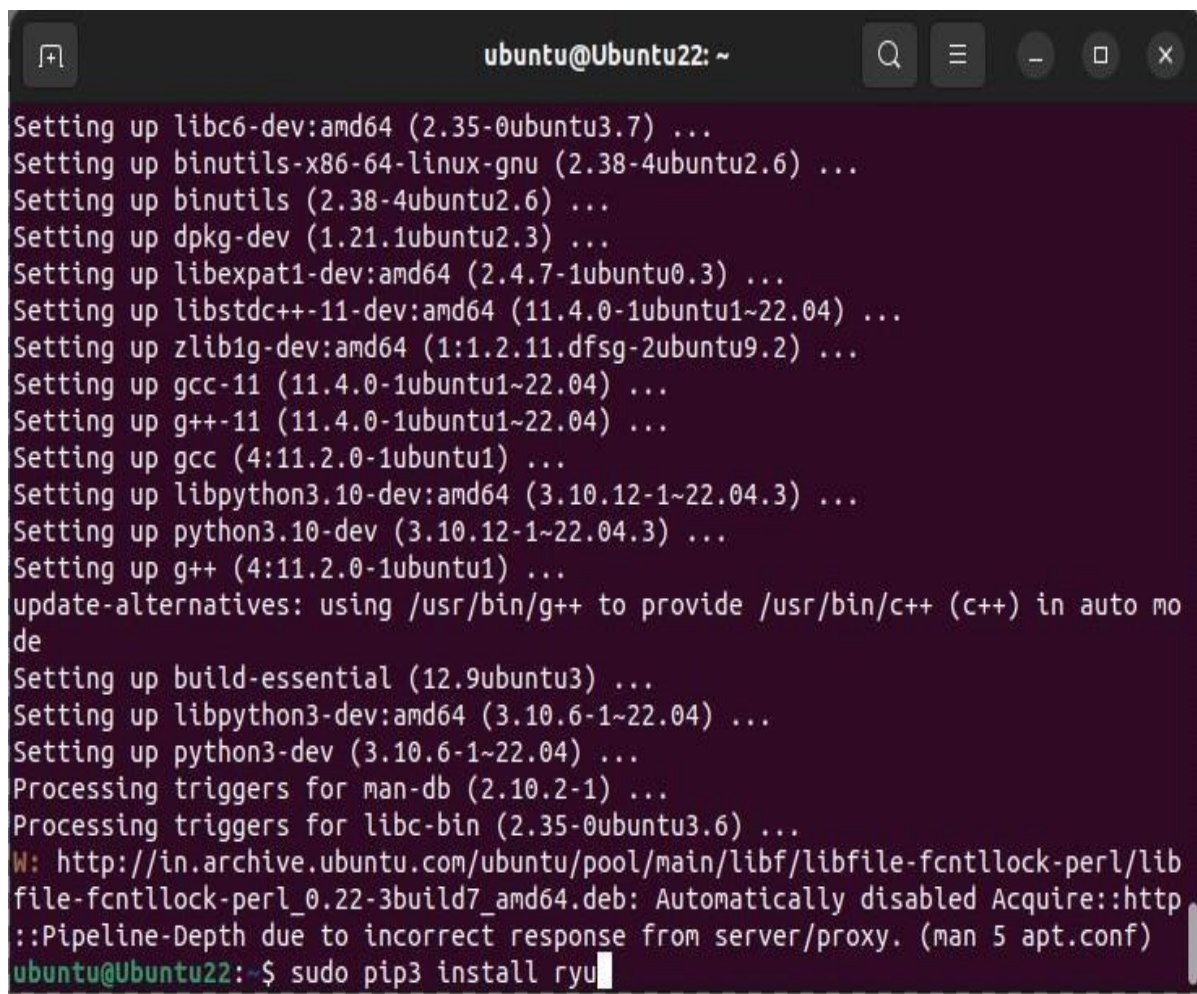
A terminal window titled 'ubuntu@Ubuntu22: ~' with search, menu, and window control icons. The command 'ubuntu@Ubuntu22:~\$ sudo apt install python3-pip' is entered and the cursor is at the end of the line.

```
ubuntu@Ubuntu22:~$ sudo apt install python3-pip
```

Step 2:

Install the ryu controller using the command,

```
sudo pip3 install ryu
```

A terminal window titled 'ubuntu@Ubuntu22: ~' showing the output of the previous command. It lists various development packages being installed, including libc6-dev, binutils, dpkg-dev, libexpat1-dev, libstdc++-11-dev, zlib1g-dev, gcc-11, g++-11, gcc, libpython3.10-dev, python3.10-dev, and g++. It also shows the update-alternatives command being used to set /usr/bin/g++ as the default for c++. Finally, it shows the command 'ubuntu@Ubuntu22:~\$ sudo pip3 install ryu' being entered.

```
Setting up libc6-dev:amd64 (2.35-0ubuntu3.7) ...
Setting up binutils-x86-64-linux-gnu (2.38-4ubuntu2.6) ...
Setting up binutils (2.38-4ubuntu2.6) ...
Setting up dpkg-dev (1.21.1ubuntu2.3) ...
Setting up libexpat1-dev:amd64 (2.4.7-1ubuntu0.3) ...
Setting up libstdc++-11-dev:amd64 (11.4.0-1ubuntu1~22.04) ...
Setting up zlib1g-dev:amd64 (1:1.2.11.dfsg-2ubuntu9.2) ...
Setting up gcc-11 (11.4.0-1ubuntu1~22.04) ...
Setting up g++-11 (11.4.0-1ubuntu1~22.04) ...
Setting up gcc (4:11.2.0-1ubuntu1) ...
Setting up libpython3.10-dev:amd64 (3.10.12-1~22.04.3) ...
Setting up python3.10-dev (3.10.12-1~22.04.3) ...
Setting up g++ (4:11.2.0-1ubuntu1) ...
update-alternatives: using /usr/bin/g++ to provide /usr/bin/c++ (c++) in auto mo
de
Setting up build-essential (12.9ubuntu3) ...
Setting up libpython3-dev:amd64 (3.10.6-1~22.04) ...
Setting up python3-dev (3.10.6-1~22.04) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.6) ...
W: http://in.archive.ubuntu.com/ubuntu/pool/main/libf/libfile-fcntllock-perl/lib
file-fcntllock-perl_0.22-3build7_amd64.deb: Automatically disabled Acquire::http
::Pipeline-Depth due to incorrect response from server/proxy. (man 5 apt.conf)
ubuntu@Ubuntu22:~$ sudo pip3 install ryu
```

Step 3:

Install the mininet in Ubuntu using the following commands,

`sudo apt install git`

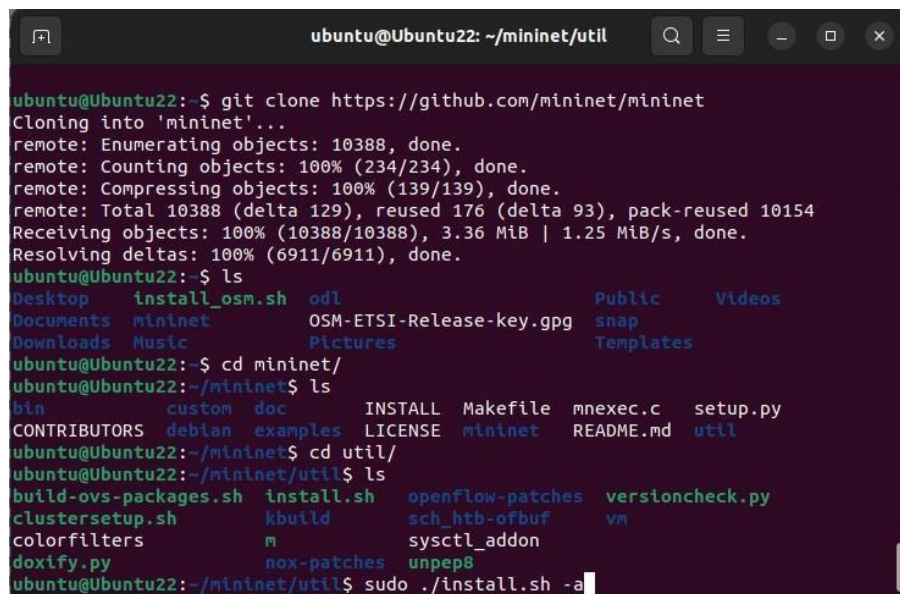
```
ubuntu@Ubuntu22:~$ sudo apt install git
```

`git clone https://github.com/mininet/mininet`

`cd mininet`

`cd util`

`sudo ./install.sh -a`



```
ubuntu@Ubuntu22: ~/mininet/util
ubuntu@Ubuntu22:~$ git clone https://github.com/mininet/mininet
Cloning into 'mininet'...
remote: Enumerating objects: 10388, done.
remote: Counting objects: 100% (234/234), done.
remote: Compressing objects: 100% (139/139), done.
remote: Total 10388 (delta 129), reused 176 (delta 93), pack-reused 10154
Receiving objects: 100% (10388/10388), 3.36 MiB | 1.25 MiB/s, done.
Resolving deltas: 100% (6911/6911), done.
ubuntu@Ubuntu22:~$ ls
Desktop  install_osm.sh  odl  Public  Videos
Documents  mininet  OSM-ETSI-Release-key.gpg  snap
Downloads  Music  Pictures  Templates
ubuntu@Ubuntu22:~$ cd mininet/
ubuntu@Ubuntu22:~/mininet$ ls
bin      custom  doc      INSTALL  Makefile  mnexec.c  setup.py
CONTRIBUTORS  debian  examples  LICENSE  mininet  README.md  util
ubuntu@Ubuntu22:~/mininet$ cd util/
ubuntu@Ubuntu22:~/mininet/util$ ls
build-ovs-packages.sh  install.sh  openflow-patches  versioncheck.py
clusterstartup.sh      kbuild     sch_htb-ofbuf     vm
colorfilters           m          sysctl_addon
doxify.py              nox-patches  unpep8
ubuntu@Ubuntu22:~/mininet/util$ sudo ./install.sh -a
```

Step 4: Create the mininet network.

Create the mininet network using the command,

`sudo mn`

```
ubuntu@Ubuntu22: ~/mininet/util
make[2]: Leaving directory '/home/ubuntu/oflops/cbench'
make[1]: Leaving directory '/home/ubuntu/oflops/cbench'
Making install in doc
make[1]: Entering directory '/home/ubuntu/oflops/doc'
make[1]: Nothing to be done for 'install'.
make[1]: Leaving directory '/home/ubuntu/oflops/doc'
Enjoy Mininet!
ubuntu@Ubuntu22:~/mininet/util$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

Check whether the network present and the host reachability using the pingall command.

Pingall

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 60.708 seconds
ubuntu@Ubuntu22:~/mininet/util$
```

Step 5: Create a simple mininet topology.

The simple mininet topology is created by using the following command.

`sudo mn --topo single,3 --mac --switch ovsk`

```
node placement for cluster (experiment)
ubuntu@Ubuntu22:~/mininet/util$ sudo mn --topo single,3 --mac --switch ovsk
--controller remote
*** Creating network
*** Adding controller
Unable to contact the remote controller at 127.0.0.1:6653
Unable to contact the remote controller at 127.0.0.1:6633
Setting remote controller to 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

Check whether the network present and the host reachability using the pingall command.

Pingall

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet>
```

Step 6: Configure hosts in network.

Configuring the hosts in the created simple mininet network using the following commands,

xterm h1 h2 h3

```
mininet> xterm h1 h2 h3
mininet>
```

In the node:h1, put

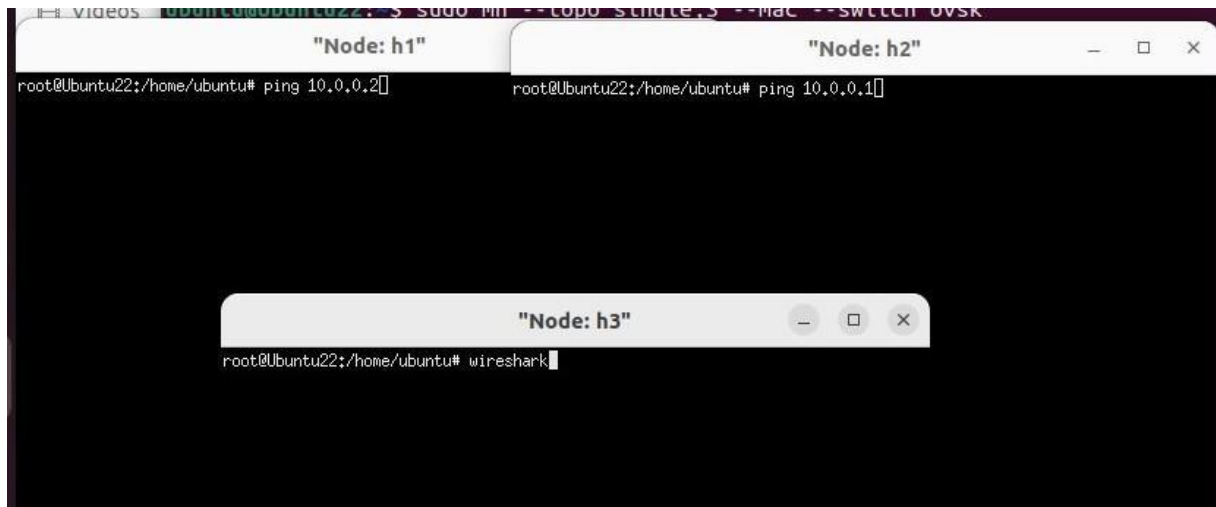
ping 10.0.0.2

In the Node:h2, put

Ping 10.0.0.1

And in the Node:h3, put a command

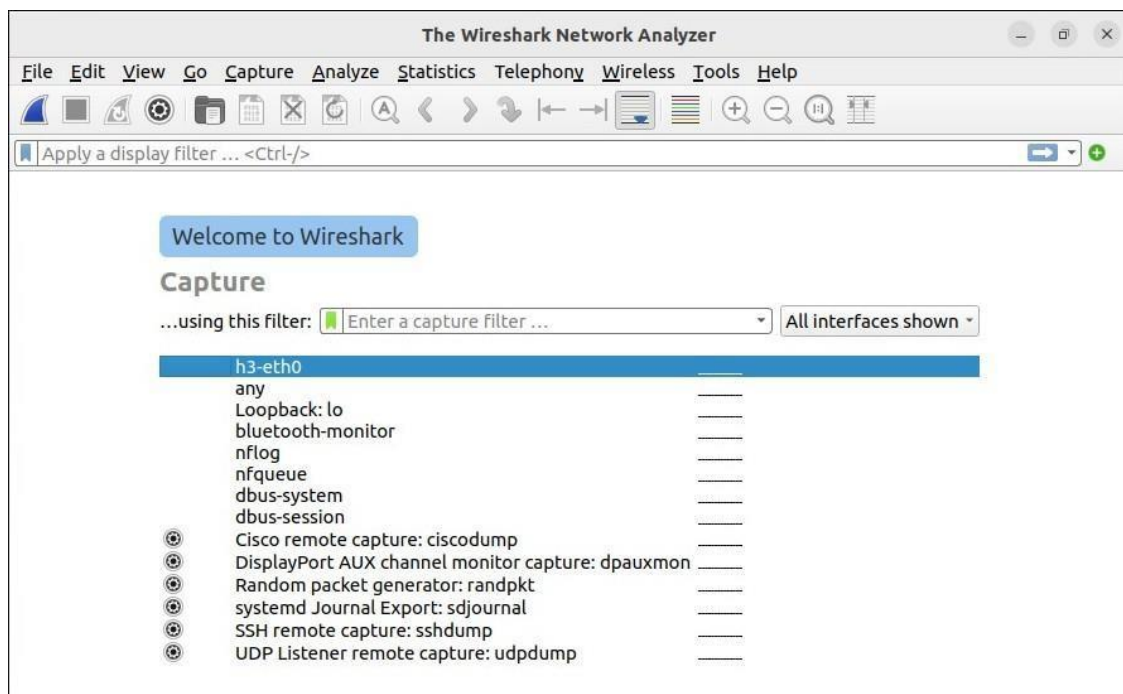
wireshark



Step 7: Use Wireshark to capture and visualize OpenFlow messages.

Using the command **wireshark** in the Node:h3 the wireshark will start to capture the packets as shown below.

- a) In the wireshark welcome window double click the **h3-eth0**



And then the packet will be started to capture as shown below.

Ubuntu 22.04 [Running] - Oracle VM VirtualBox

File Machine View Input Devices Help

Activities Wireshark Apr 26 08:17

Capturing from h3-eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	fe80::4010:e6ff:fe6...	ff02::fb	MDNS	107	Standard query 0x00
2	33.584852107	fe80::4010:e6ff:fe6...	ff02::2	ICMPv6	70	Router Solicitation
3	33.587712376	fe80::200:ff:fe00:2	ff02::2	ICMPv6	70	Router Solicitation
4	49.963441161	fe80::200:ff:fe00:3	ff02::2	ICMPv6	70	Router Solicitation
5	66.356971730	fe80::200:ff:fe00:1	ff02::2	ICMPv6	70	Router Solicitation

Frame 1: 107 bytes on wire (856 bits), 107 bytes captured (856 bits) on interface h3-eth0, id 0

Ethernet II, Src: 42:10:e6:6b:3c:d8 (42:10:e6:6b:3c:d8), Dst: IPv6mcast_fb (33:33:00:00:00:fb)

Internet Protocol Version 6, Src: fe80::4010:e6ff:fe6b:3cd8, Dst: ff02::fb

User Datagram Protocol, Src Port: 5353, Dst Port: 5353

Multicast Domain Name System (query)

```

0000  33 33 00 00 00 fb 42 10 e6 6b 3c d8 86 dd 60 03  33...B.k<...
0010  4f ac 00 35 11 ff fe 80 00 00 00 00 00 40 10  0.5...@.
0020  e6 ff fe 6b 3c d8 ff 02 00 00 00 00 00 00 00  ...k<...
0030  00 00 00 00 00 fb 14 e9 14 e9 00 35 61 19 00 00  ...5a...
0040  00 00 00 02 00 00 00 00 00 00 05 5f 69 70 70 73  ..._ipps
0050  04 5f 74 63 70 05 6c 6f 63 61 6c 00 00 0c 00 01  _tcp.lo cal...
0060  04 5f 69 70 70 c0 12 00 0c 00 01  _ipp...

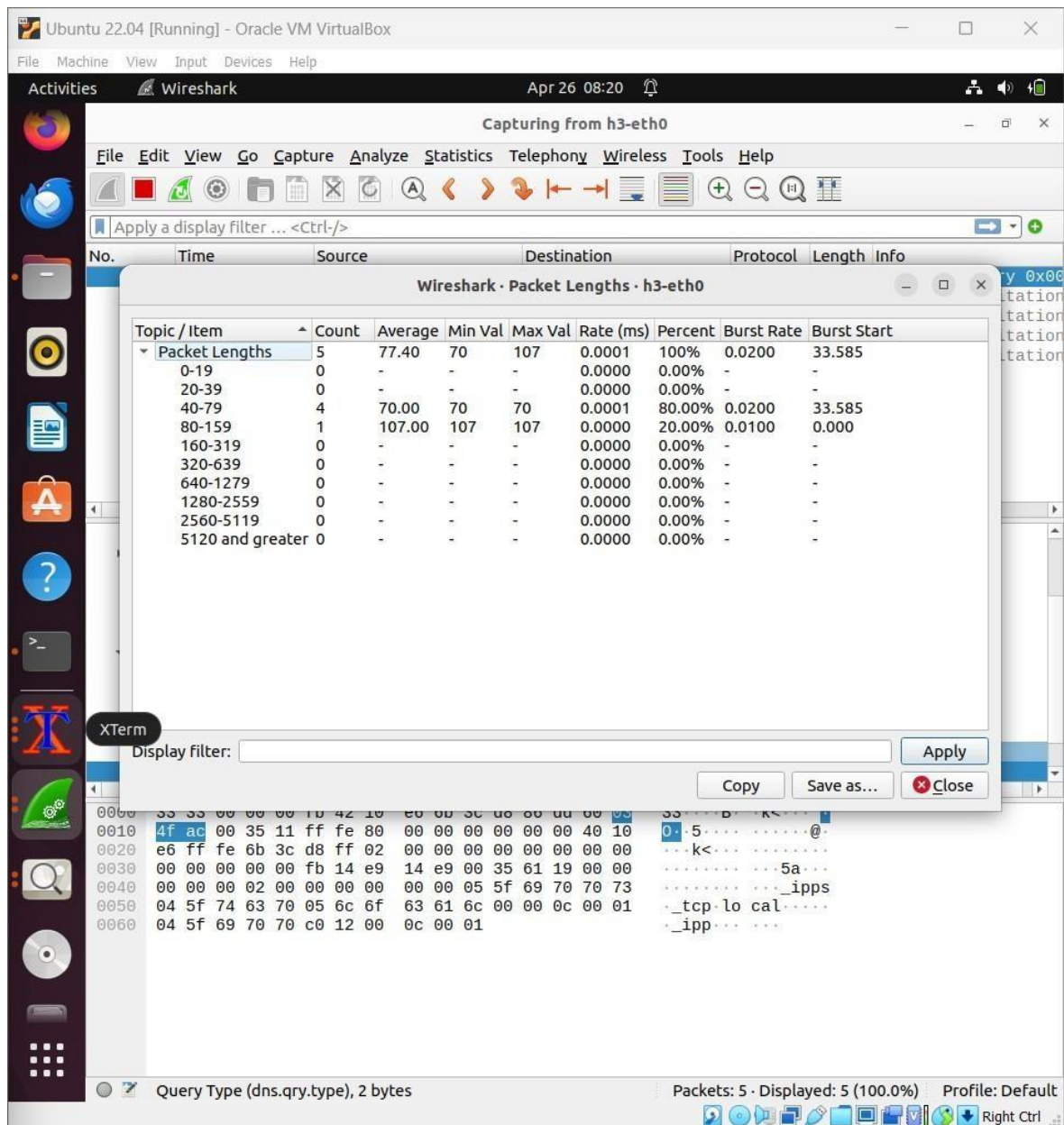
```

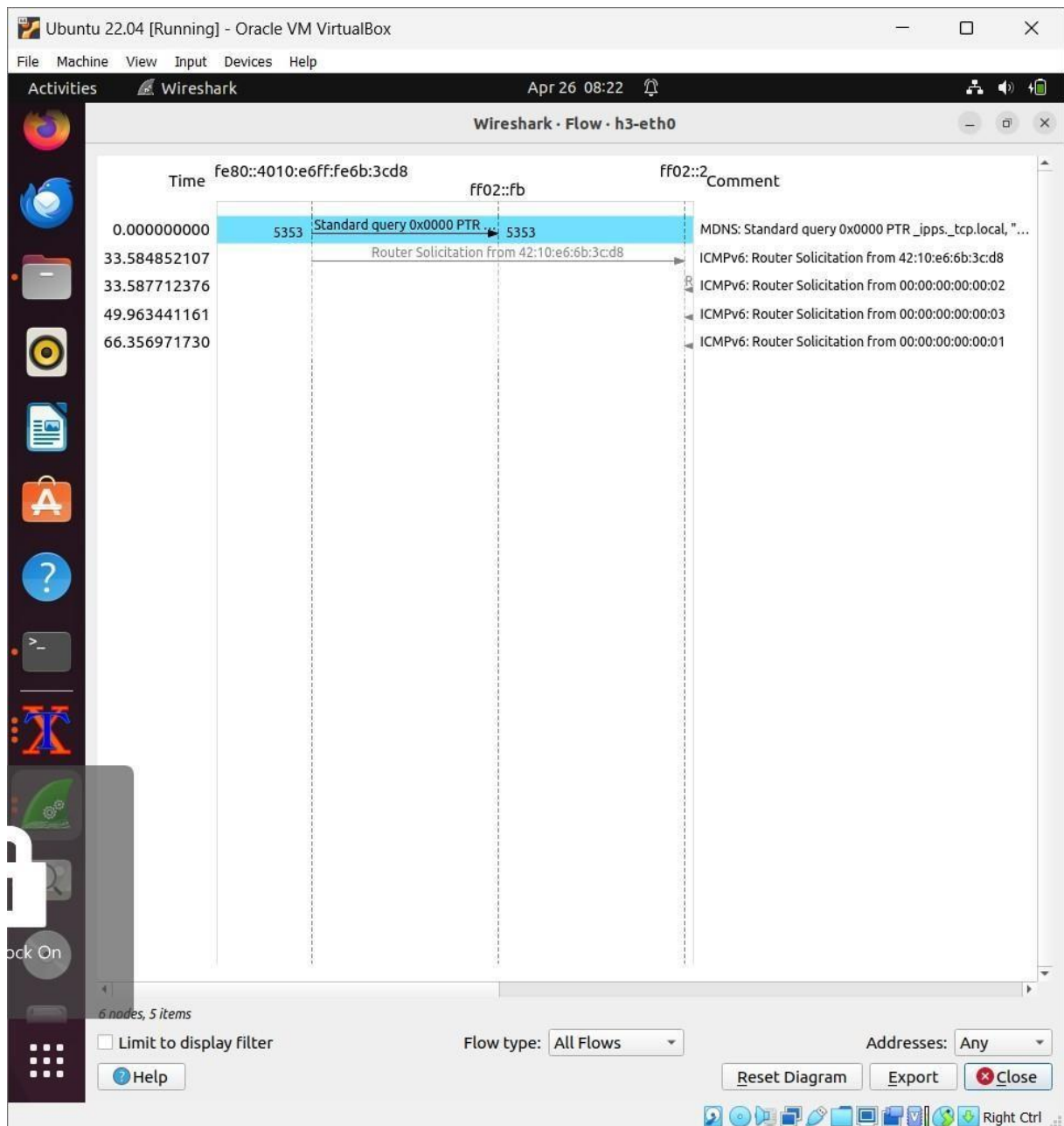
h3-eth0: <live capture in progress>

Packets: 5 · Displayed: 5 (100.0%) Profile: Default

Right Ctrl

- b) Through the statistics tab we may see the PACKET IN and PACKET OUT and also FLOW MOD as shown below.





Result:

Thus the simple mininet topology with SDN controller was created successfully. And the Wireshark was used successfully to capture and visualize the OpenFlow messages such as OpenFlow FLOW MOD, PACKET IN, PACKET OUT etc.

Ex. No: 3 DATE:	Create a SDN application that uses the Northbound API to program flow table rules on the switch for various use cases like L2 learning switch, Traffic Engineering, Firewall etc.
----------------------------------	--

AIM:

To create a SDN application that uses the Northbound API to program flow table rules on the switch for various use cases like L2 learning switch, Traffic Engineering, Firewall.

ALGORITHM:

Step 1: Open the Ubuntu 20.04.6 in the Oracle VM Virtual Box Manager.

And install the dependencies.

Step 2: Install the Ryu controller properly.

Step 3: Downgrade the eventlet.

Step 4: Install the FlowManager (SDN application).

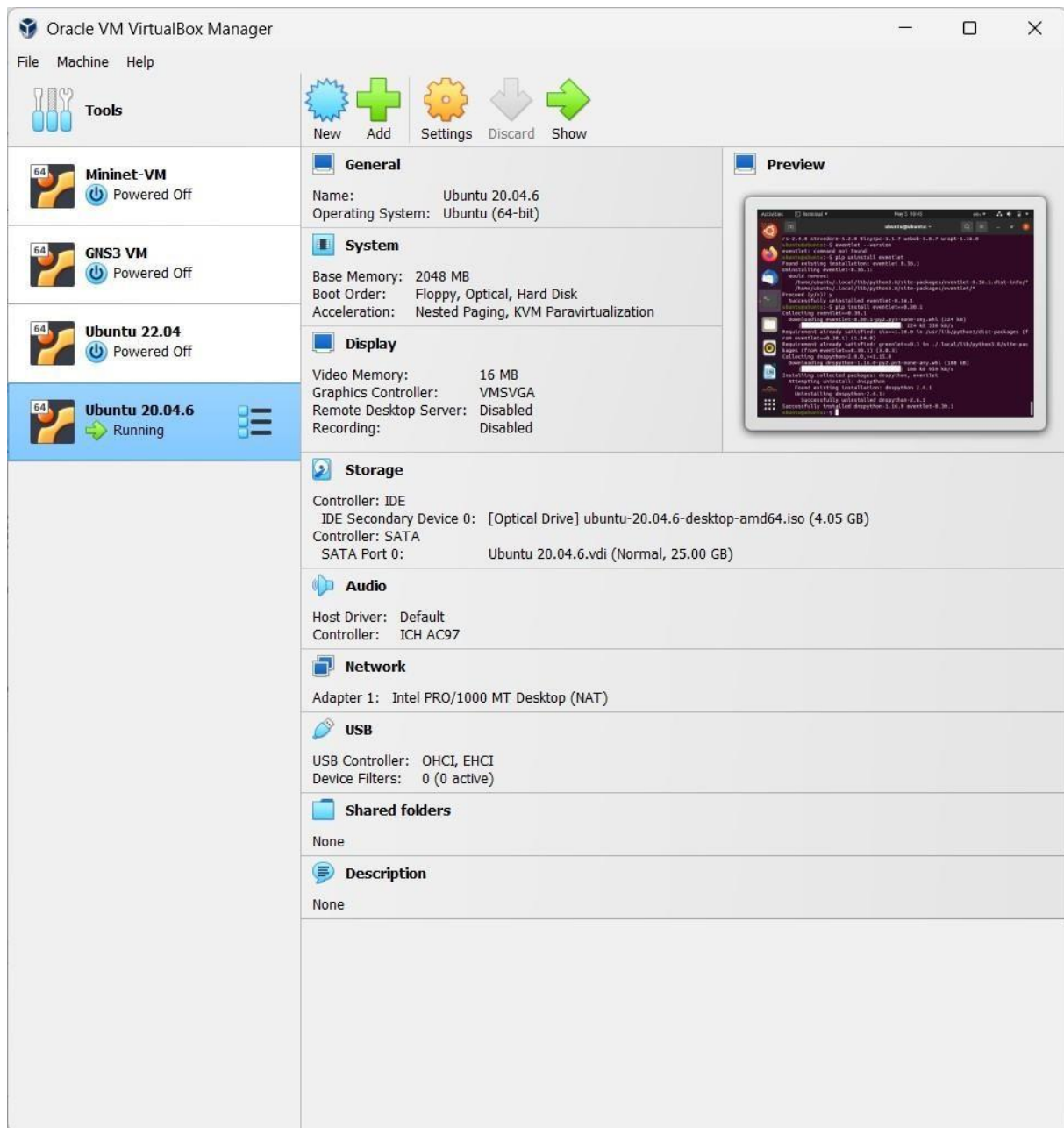
Step 5: Run the FlowManager application.

Step 6: Use a web browser to launch the site.

PROCEDURE:

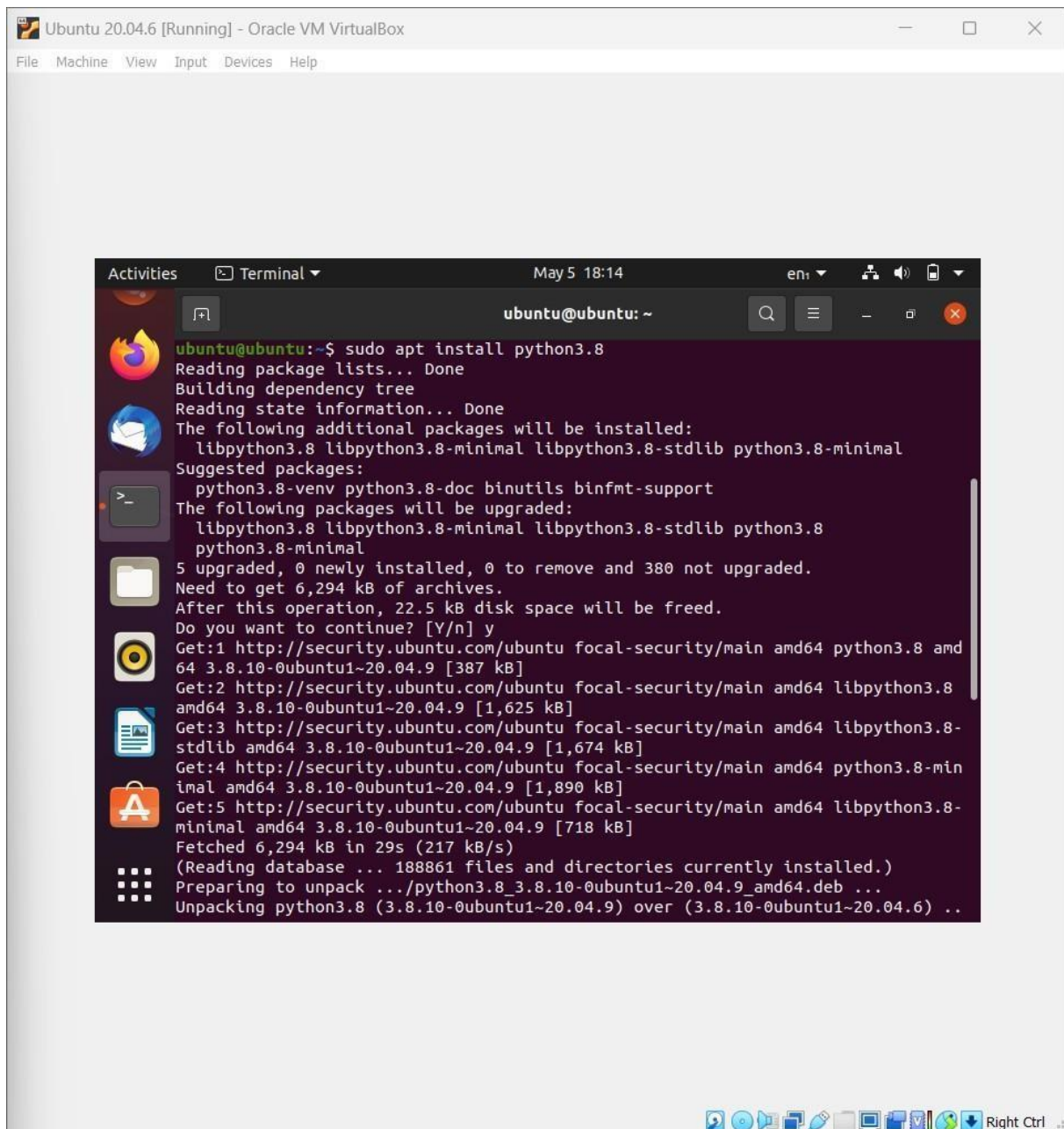
Step 1:

Open the Ubuntu 20.04.6 in the Oracle VM Virtual Box Manager.



And install the dependencies.

A) Installing python 3.8 in the ubuntu 20.04.6.



B) Install the pip.

1. To install pip enable the universe repository first.

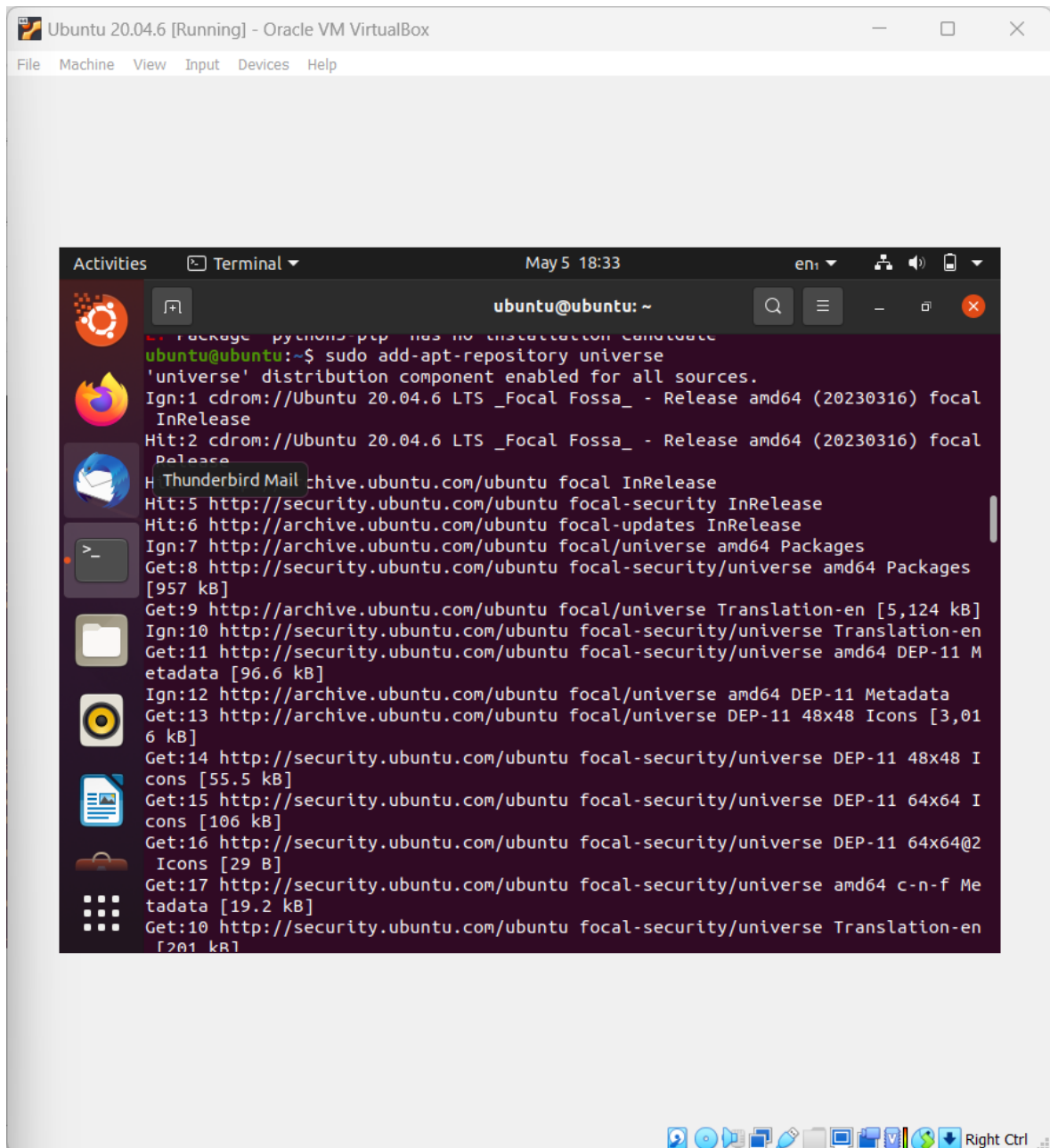
```
sudo add-apt-repository universe
```

2. Then run the following command to synchronize the package database.

```
sudo apt update
```

3. Finally install pip3 with the following command.

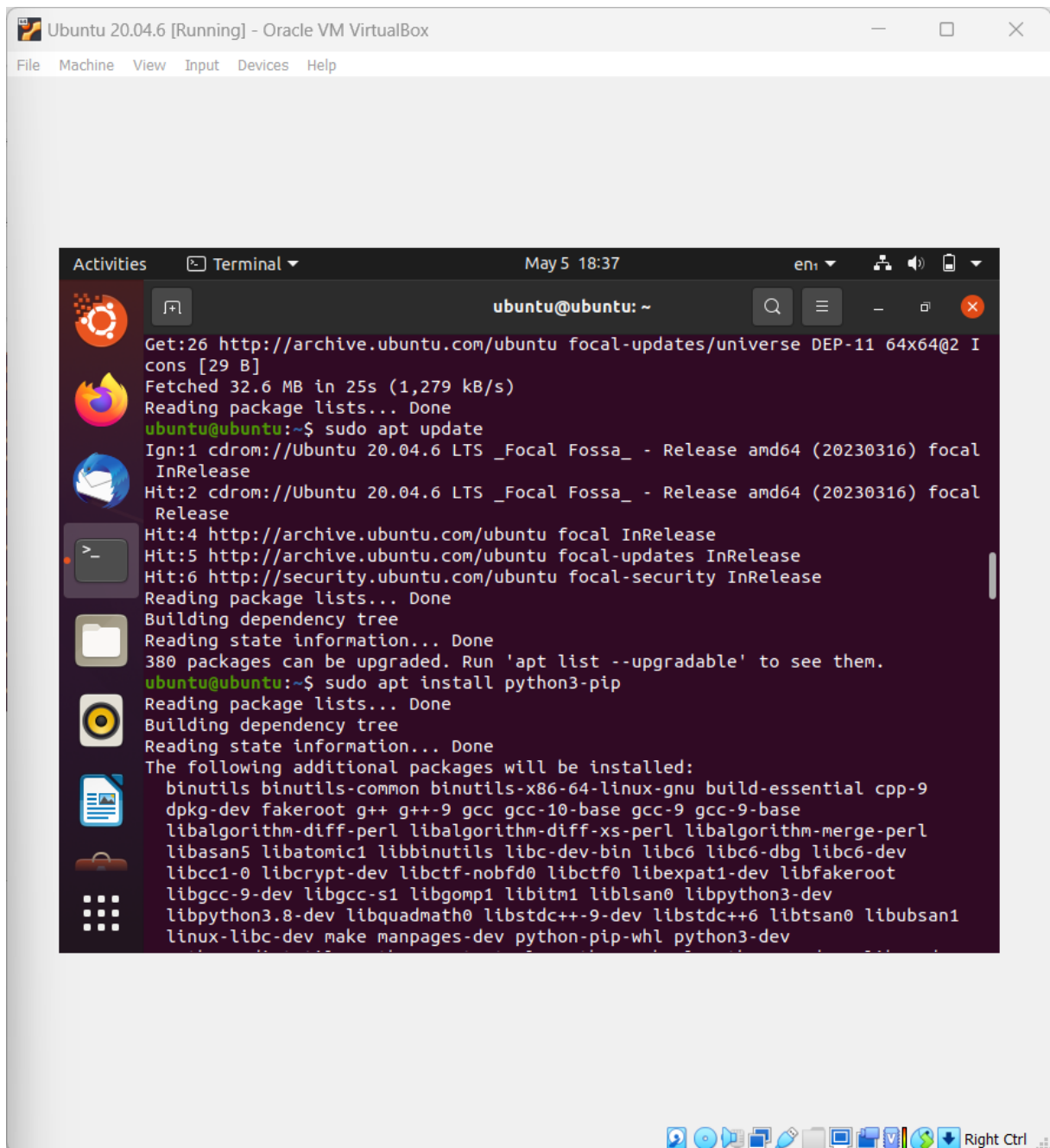
sudo apt install python3-pip



```
Ubuntu 20.04.6 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

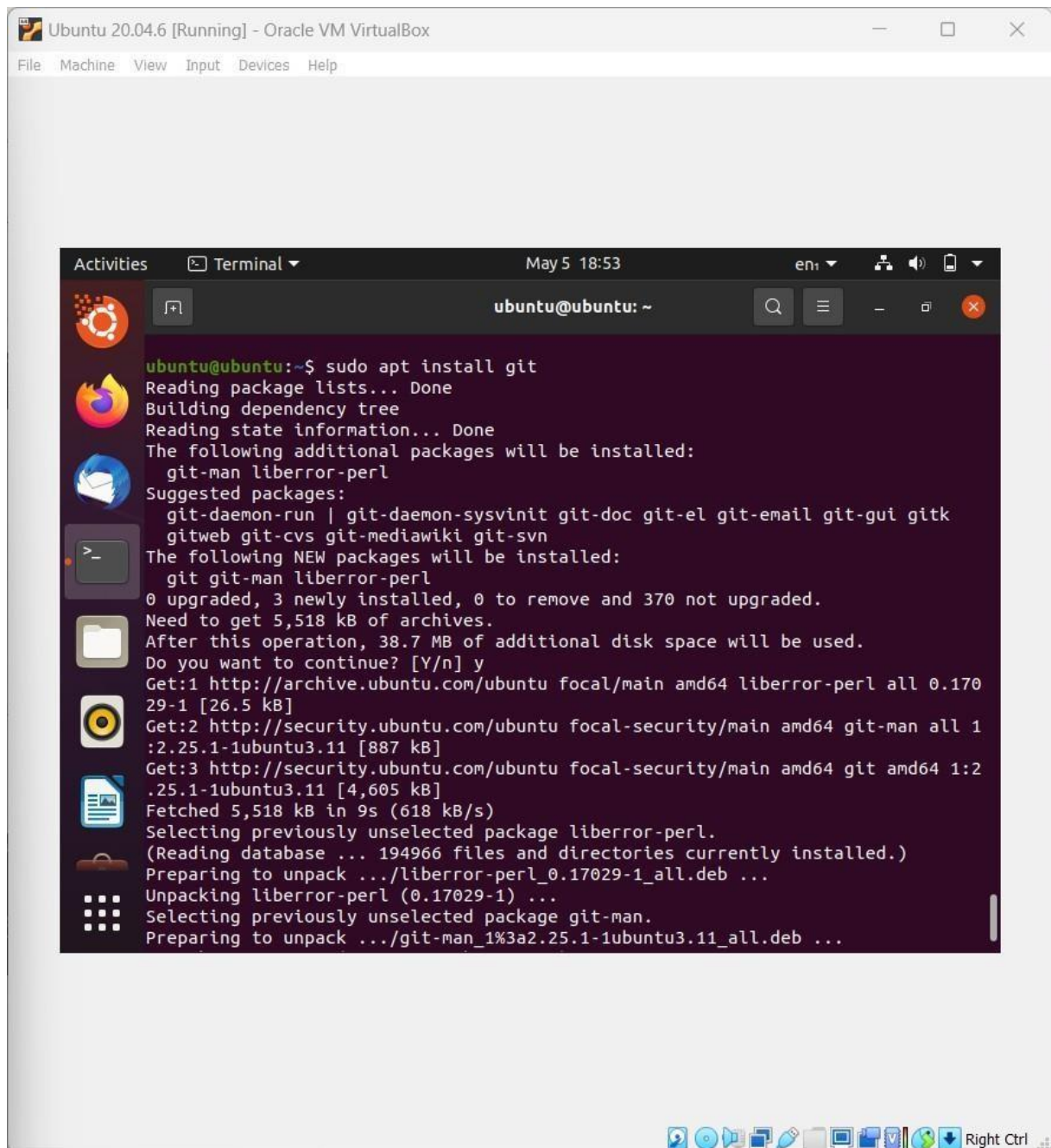
Activities Terminal May 5 18:33 en1
ubuntu@ubuntu: ~

E: Package python3-pip has no installation candidate
ubuntu@ubuntu:~$ sudo add-apt-repository universe
'universe' distribution component enabled for all sources.
Ign:1 cdrom://Ubuntu 20.04.6 LTS _Focal Fossa_ - Release amd64 (20230316) focal
InRelease
Hit:2 cdrom://Ubuntu 20.04.6 LTS _Focal Fossa_ - Release amd64 (20230316) focal
Release
Hit:3 cdrom://Ubuntu 20.04.6 LTS _Focal Fossa_ - Release amd64 (20230316) focal
InRelease
Hit:4 cdrom://Ubuntu 20.04.6 LTS _Focal Fossa_ - Release amd64 (20230316) focal
InRelease
Hit:5 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:6 http://archive.ubuntu.com/ubuntu focal-updates InRelease
Ign:7 http://archive.ubuntu.com/ubuntu focal/universe amd64 Packages
Get:8 http://security.ubuntu.com/ubuntu focal-security/universe amd64 Packages
[957 kB]
Get:9 http://archive.ubuntu.com/ubuntu focal/universe Translation-en [5,124 kB]
Ign:10 http://security.ubuntu.com/ubuntu focal-security/universe Translation-en
Get:11 http://security.ubuntu.com/ubuntu focal-security/universe amd64 DEP-11 M
etadata [96.6 kB]
Ign:12 http://archive.ubuntu.com/ubuntu focal/universe amd64 DEP-11 Metadata
Get:13 http://archive.ubuntu.com/ubuntu focal/universe DEP-11 48x48 Icons [3,01
6 kB]
Get:14 http://security.ubuntu.com/ubuntu focal-security/universe DEP-11 48x48 I
cons [55.5 kB]
Get:15 http://security.ubuntu.com/ubuntu focal-security/universe DEP-11 64x64 I
cons [106 kB]
Get:16 http://security.ubuntu.com/ubuntu focal-security/universe DEP-11 64x64@2
Icons [29 B]
Get:17 http://security.ubuntu.com/ubuntu focal-security/universe amd64 c-n-f Me
tadata [19.2 kB]
Get:18 http://security.ubuntu.com/ubuntu focal-security/universe Translation-en
[201 kB]
```

C) Install git.

`sudo apt install git`

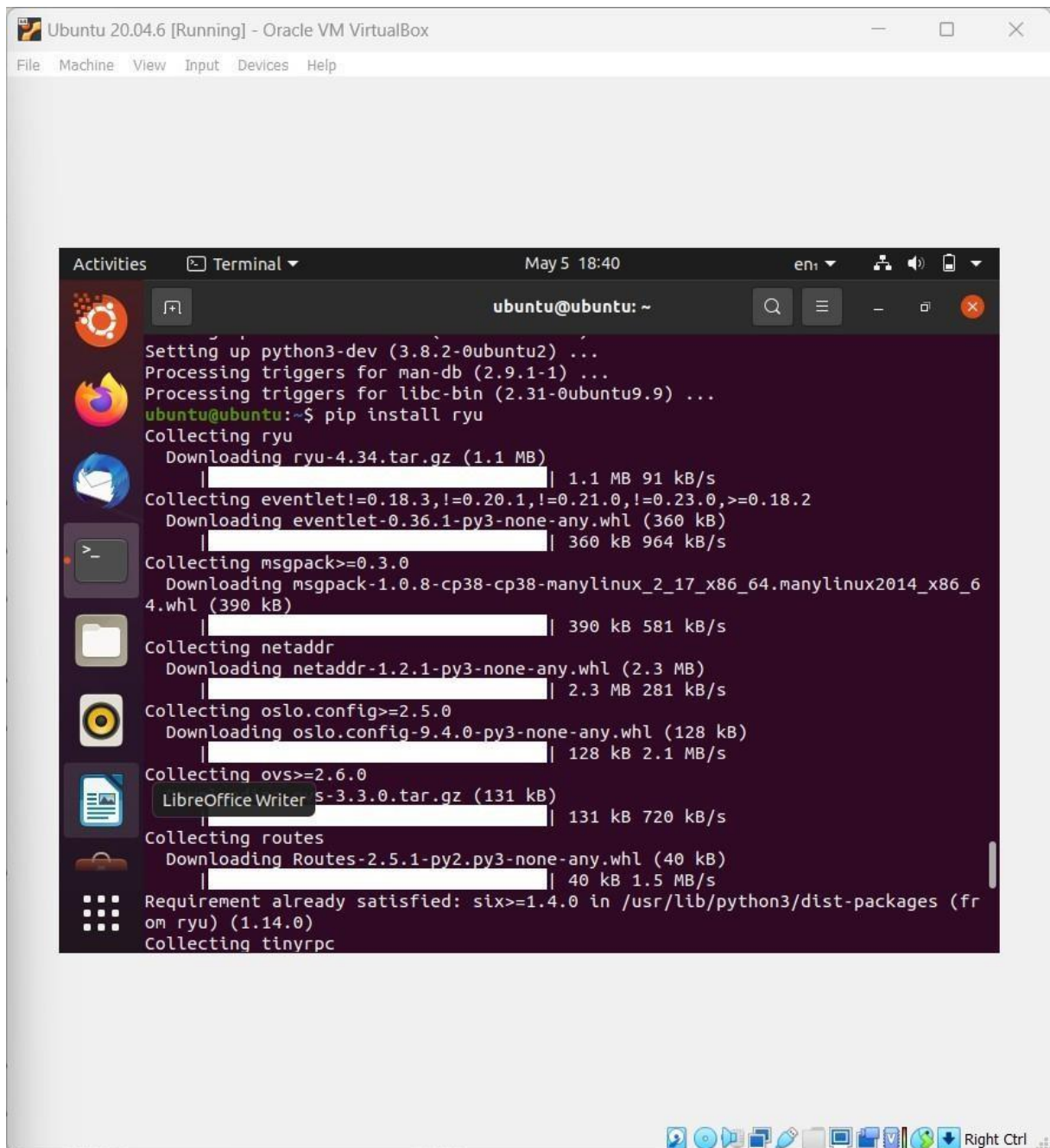


Step 2:

Install the Ryu controller properly.

By using the command,

`pip install ryu`



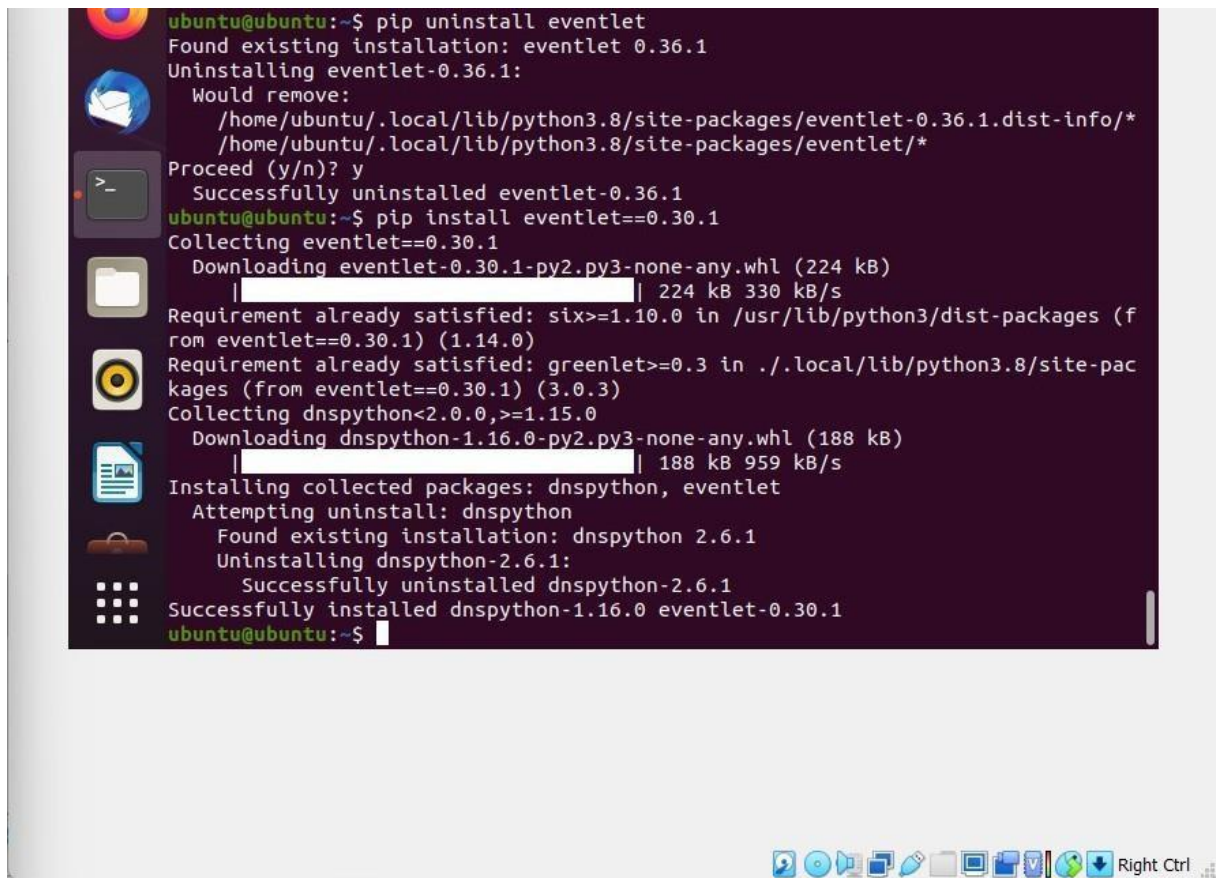
Step 3:

Downgrade the eventlet for smooth running of the application.

By using the following commands,

pip uninstall eventlet

pip install eventlet==0.30.1

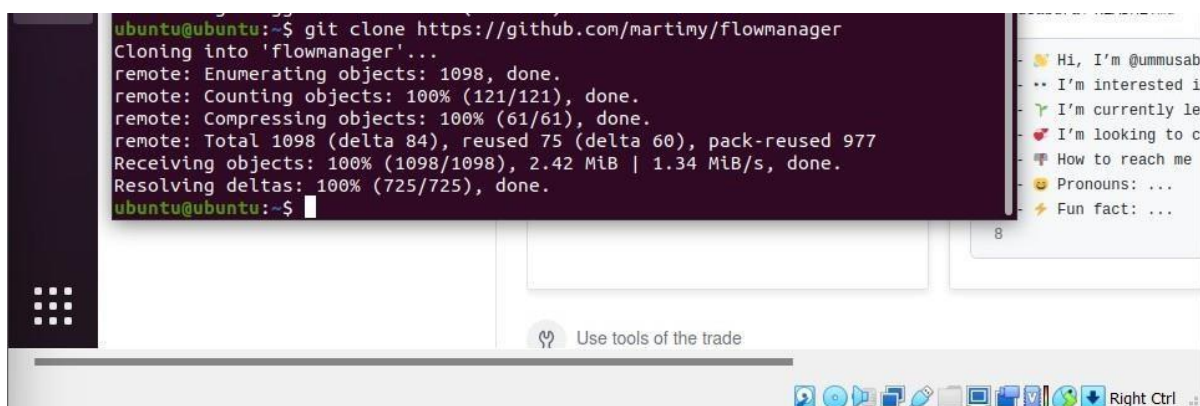


```
ubuntu@ubuntu:~$ pip uninstall eventlet
Found existing installation: eventlet 0.36.1
Uninstalling eventlet-0.36.1:
  Would remove:
    /home/ubuntu/.local/lib/python3.8/site-packages/eventlet-0.36.1.dist-info/*
    /home/ubuntu/.local/lib/python3.8/site-packages/eventlet/*
Proceed (y/n)? y
  Successfully uninstalled eventlet-0.36.1
ubuntu@ubuntu:~$ pip install eventlet==0.30.1
Collecting eventlet==0.30.1
  Downloading eventlet-0.30.1-py2.py3-none-any.whl (224 kB)
    |#####| 224 kB 330 kB/s
Requirement already satisfied: six>=1.10.0 in /usr/lib/python3/dist-packages (from eventlet==0.30.1) (1.14.0)
Requirement already satisfied: greenlet>=0.3 in ~/.local/lib/python3.8/site-packages (from eventlet==0.30.1) (3.0.3)
Collecting dnspython<2.0.0,>=1.15.0
  Downloading dnspython-1.16.0-py2.py3-none-any.whl (188 kB)
    |#####| 188 kB 959 kB/s
Installing collected packages: dnspython, eventlet
  Attempting uninstall: dnspython
    Found existing installation: dnspython 2.6.1
    Uninstalling dnspython-2.6.1:
      Successfully uninstalled dnspython-2.6.1
  Successfully installed dnspython-1.16.0 eventlet-0.30.1
ubuntu@ubuntu:~$
```

Step 4:

Install the FlowManager (SDN application) using the following command.

`git clone https://github.com/martimy/flowmanager`



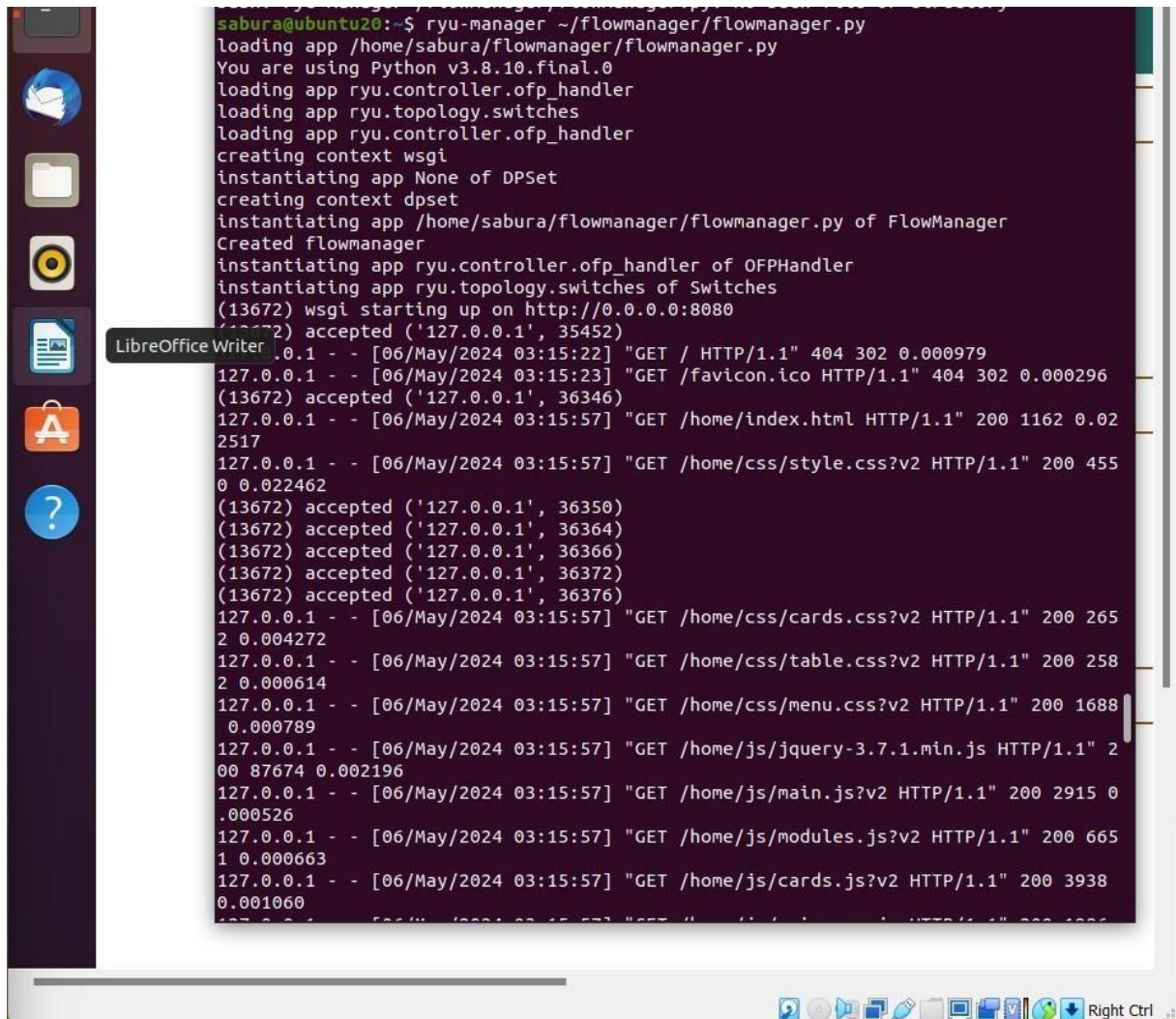
```
ubuntu@ubuntu:~$ git clone https://github.com/martimy/flowmanager
Cloning into 'flowmanager'...
remote: Enumerating objects: 1098, done.
remote: Counting objects: 100% (121/121), done.
remote: Compressing objects: 100% (61/61), done.
remote: Total 1098 (delta 84), reused 75 (delta 60), pack-reused 977
Receiving objects: 100% (1098/1098), 2.42 MiB | 1.34 MiB/s, done.
Resolving deltas: 100% (725/725), done.
ubuntu@ubuntu:~$
```

Step 5:

Run the FlowManager app.

Run the FlowManager

ryu-manager ~/flowmanager/flowmanager.py



```
sabura@ubuntu20:~$ ryu-manager ~/flowmanager/flowmanager.py
loading app /home/sabura/flowmanager/flowmanager.py
You are using Python v3.8.10.final.0
loading app ryu.controller.ofp_handler
loading app ryu.topology.switches
loading app ryu.controller.ofp_handler
creating context wsgi
instantiating app None of DPSet
creating context dpset
instantiating app /home/sabura/flowmanager/flowmanager.py of FlowManager
Created flowmanager
instantiating app ryu.controller.ofp_handler of OFPHandler
instantiating app ryu.topology.switches of Switches
(13672) wsgi starting up on http://0.0.0.0:8080
(13672) accepted ('127.0.0.1', 35452)
127.0.0.1 - - [06/May/2024 03:15:22] "GET / HTTP/1.1" 404 302 0.000979
127.0.0.1 - - [06/May/2024 03:15:23] "GET /favicon.ico HTTP/1.1" 404 302 0.000296
(13672) accepted ('127.0.0.1', 36346)
127.0.0.1 - - [06/May/2024 03:15:57] "GET /home/index.html HTTP/1.1" 200 1162 0.02
2517
127.0.0.1 - - [06/May/2024 03:15:57] "GET /home/css/style.css?v2 HTTP/1.1" 200 455
0 0.022462
(13672) accepted ('127.0.0.1', 36350)
(13672) accepted ('127.0.0.1', 36364)
(13672) accepted ('127.0.0.1', 36366)
(13672) accepted ('127.0.0.1', 36372)
(13672) accepted ('127.0.0.1', 36376)
127.0.0.1 - - [06/May/2024 03:15:57] "GET /home/css/cards.css?v2 HTTP/1.1" 200 265
2 0.004272
127.0.0.1 - - [06/May/2024 03:15:57] "GET /home/css/table.css?v2 HTTP/1.1" 200 258
2 0.000614
127.0.0.1 - - [06/May/2024 03:15:57] "GET /home/css/menu.css?v2 HTTP/1.1" 200 1688
0.000789
127.0.0.1 - - [06/May/2024 03:15:57] "GET /home/js/jquery-3.7.1.min.js HTTP/1.1" 2
00 87674 0.002196
127.0.0.1 - - [06/May/2024 03:15:57] "GET /home/js/main.js?v2 HTTP/1.1" 200 2915 0
.000526
127.0.0.1 - - [06/May/2024 03:15:57] "GET /home/js/modules.js?v2 HTTP/1.1" 200 665
1 0.000663
127.0.0.1 - - [06/May/2024 03:15:57] "GET /home/js/cards.js?v2 HTTP/1.1" 200 3938
0.001060
127.0.0.1 - - [06/May/2024 03:15:57] "GET /home/js/cards.js?v2 HTTP/1.1" 200 3938
0.001060
```

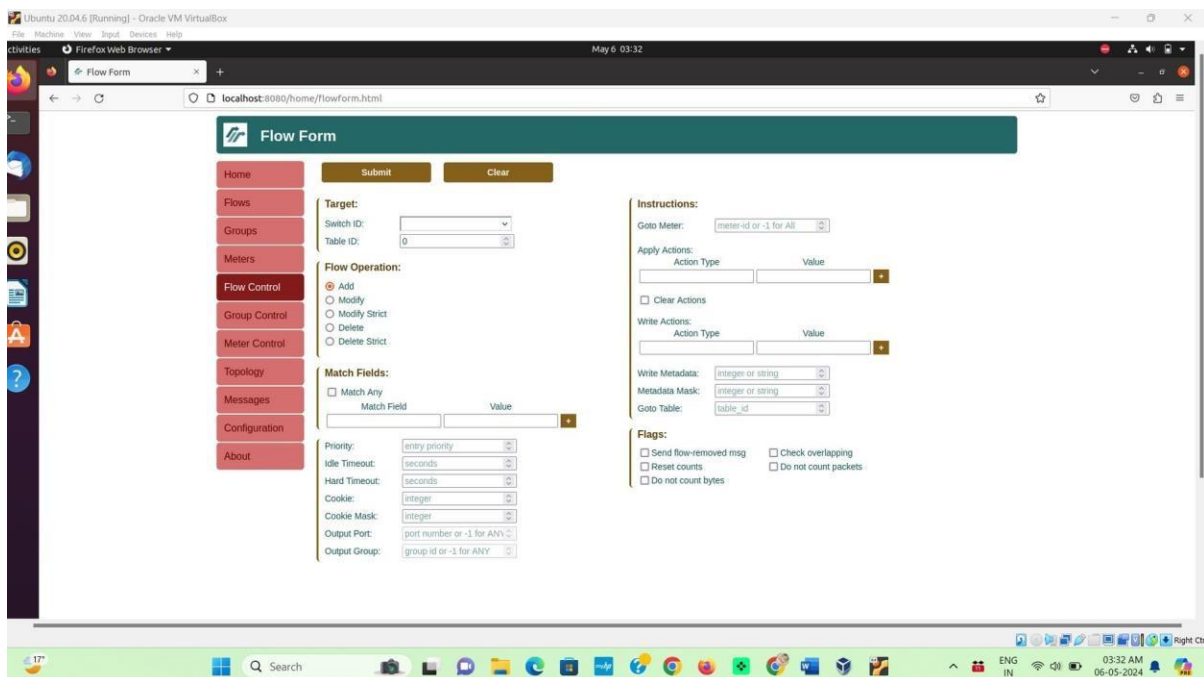
The following command is used to display the topology:

ryu-manager --observe-links ~/flowmanager/flowmanager.py ryu.app.simple_switch_13

```
sabura@ubuntu20:~$ ryu-manager --observe-links ~/flowmanager/flowmanager.py ryu.app.simple_switch_13
loading app /home/sabura/flowmanager/flowmanager.py
You are using Python v3.8.10.final.0
loading app ryu.app.simple_switch_13
loading app ryu.controller.ofp_handler
loading app ryu.topology.switches
loading app ryu.controller.ofp_handler
creating context wsgi
instantiating app None of DPSet
creating context dpset
instantiating app /home/sabura/flowmanager/flowmanager.py of FlowManager
Created Flowmanager
instantiating app ryu.app.simple_switch_13 of SimpleSwitch13
instantiating app ryu.controller.ofp_handler of OFPHandler
instantiating app ryu.topology.switches of Switches
(14207) wsgi starting up on http://0.0.0.0:8080
(14207) accepted ('127.0.0.1', 49758)
127.0.0.1 - - [06/May/2024 03:27:51] "GET /data?list=switches HTTP/1.1" 200 165 0.001858
127.0.0.1 - - [06/May/2024 03:27:51] "GET /data?portdesc=1 HTTP/1.1" 200 167 0.000386
127.0.0.1 - - [06/May/2024 03:27:51] "GET /data?portstat=1 HTTP/1.1" 200 167 0.000289
127.0.0.1 - - [06/May/2024 03:27:51] "GET /data?flowsumm=1 HTTP/1.1" 200 167 0.000377
127.0.0.1 - - [06/May/2024 03:27:51] "GET /data?tablestat=1 HTTP/1.1" 200 167 0.000244
127.0.0.1 - - [06/May/2024 03:28:06] "GET /data?list=switches HTTP/1.1" 200 165 0.000854
127.0.0.1 - - [06/May/2024 03:28:06] "GET /data?portdesc=1 HTTP/1.1" 200 167 0.000361
127.0.0.1 - - [06/May/2024 03:28:06] "GET /data?portstat=1 HTTP/1.1" 200 167 0.000182
127.0.0.1 - - [06/May/2024 03:28:06] "GET /data?flowsumm=1 HTTP/1.1" 200 167 0.000237
127.0.0.1 - - [06/May/2024 03:28:06] "GET /data?tablestat=1 HTTP/1.1" 200 167 0.000172
```

Step 6:

Use a web browser to launch the site <http://localhost:8080/home/index.html>



Result:

Thus the SDN application was created and executed successfully which uses the Northbound API to program flow table rules on the switch for various use cases like L2 learning switch, Traffic Engineering, Firewall.

Ex. No: 4
DATE:

Create a simple end-to-end network service with two VNFs using vim-emu
<https://github.com/containernet/vim-emu>

AIM:

To create a simple end-to-end network service with two VNFs using vim-emu
<https://github.com/containernet/vim-emu>.

ALGORITHM:

Step 1: Start Ubuntu 22.04.6 in the Oracle Virtualbox and Deploy the vim-emu emulator.

Step 2: Build the container using docker through the following commands.

Step 3: Run the interactive container.

Step 4: Start the emulator.

PROCEDURE:

Step 1:

Deploy the vim-emu emulator using the following command.

git clone <https://osm.etsi.org/gerrit/osm/vim-emu.git>



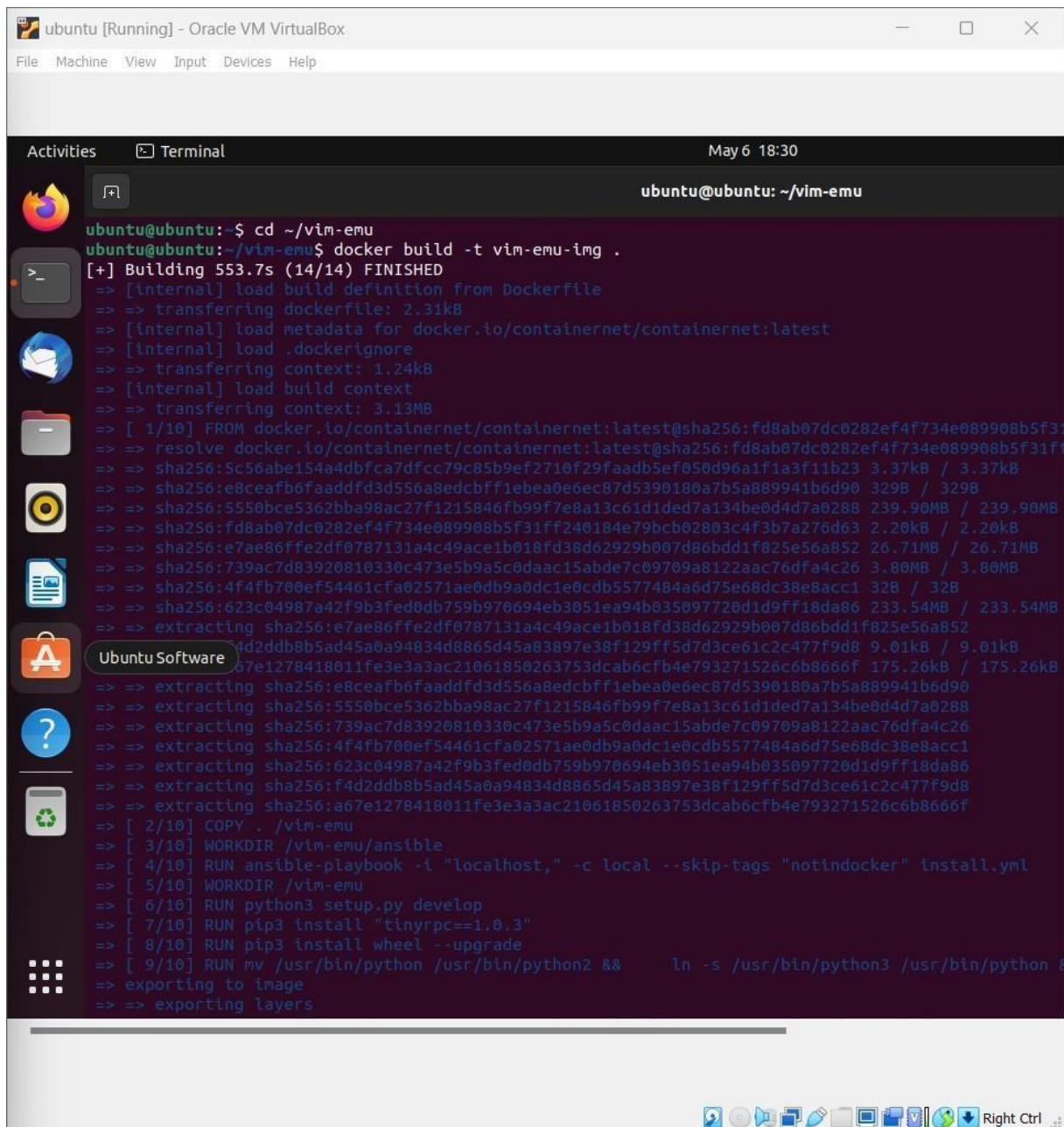
```
ubuntu@ubuntu: ~$ git clone https://github.com/containernet/containernet.git
Cloning into 'containernet'...
remote: Enumerating objects: 12472, done.
remote: Counting objects: 100% (299/299), done.
remote: Compressing objects: 100% (130/130), done.
remote: Total 12472 (delta 171), reused 286 (delta 165), pack-reused 12173
Receiving objects: 100% (12472/12472), 4.00 MiB | 656.00 KiB/s, done.
Resolving deltas: 100% (8347/8347), done.
ubuntu@ubuntu: ~$
```

Step 2:

Build the container using docker through the following commands.

cd ~/vim-emu

docker build -t vim-emu-img .



```
ubuntu [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

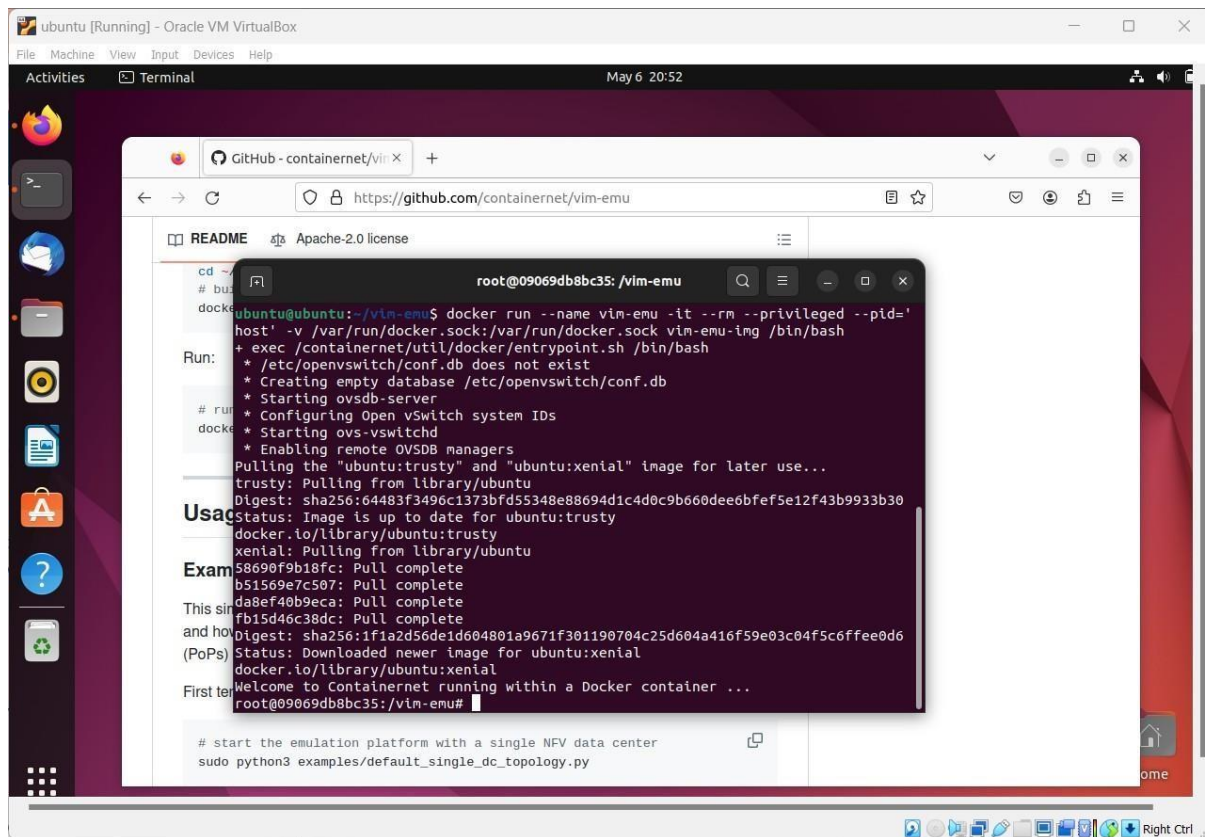
Activities Terminal May 6 18:30
ubuntu@ubuntu: ~/vim-emu

ubuntu@ubuntu:~$ cd ~/vim-emu
ubuntu@ubuntu:~/vim-emu$ docker build -t vim-emu-img .
[+] Building 553.7s (14/14) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 2.31kB
=> [internal] load metadata for docker.io/containerne...:latest
=> [internal] load .dockerignore
=> => transferring context: 1.24kB
=> [internal] load build context
=> => transferring context: 3.13MB
=> [ 1/10] FROM docker.io/containerne...:latest@sha256:fd8ab07dc0282ef4f734e089908b5f31f...
=> => resolve docker.io/containerne...:latest@sha256:fd8ab07dc0282ef4f734e089908b5f31f...
=> => sha256:5c56abe154a4dbfca7dfcc79c85b9ef2710f29faadb5ef050d96a1f1a3f11b23 3.37kB / 3.37kB
=> => sha256:e8ceafb6faaddfd3d556a8edcbff1e8a0e6ec87d5390180a7b5a889941b6d90 329B / 329B
=> => sha256:5550bce5362bba98ac27f1215846fb99f7e8a13c61d1ded7a134be0d4d7a0288 239.90MB / 239.90MB
=> => sha256:fd8ab07dc0282ef4f734e089908b5f31ff240184e79bcb02803c4f3b7a276d63 2.20kB / 2.20kB
=> => sha256:e7ae86ffe2df0787131a4c49ace1b018fd38d62929b007d86bdd1f825e56a852 26.71MB / 26.71MB
=> => sha256:739ac7d83920810330c473e5b9a5c0daac15abde7c09709a8122aac76dfa4c26 3.80MB / 3.80MB
=> => sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1 32B / 32B
=> => sha256:623c04987a42f9b3fed0db759b970694eb3051ea94b035097720d1d9ff18da86 233.54MB / 233.54MB
=> => extracting sha256:e7ae86ffe2df0787131a4c49ace1b018fd38d62929b007d86bdd1f825e56a852
=> => extracting sha256:4d2ddb8b5ad45a0a94834d8865d45a83897e38f129ff5d7d3ce61c2c477f9d8 9.01kB / 9.01kB
=> => extracting sha256:67e1278418011fe3e3a3ac21061850263753dcab6c6fb4e793271526c6b8666f 175.26kB / 175.26kB
=> => extracting sha256:e8ceafb6faaddfd3d556a8edcbff1e8a0e6ec87d5390180a7b5a889941b6d90
=> => extracting sha256:5550bce5362bba98ac27f1215846fb99f7e8a13c61d1ded7a134be0d4d7a0288
=> => extracting sha256:739ac7d83920810330c473e5b9a5c0daac15abde7c09709a8122aac76dfa4c26
=> => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1
=> => extracting sha256:623c04987a42f9b3fed0db759b970694eb3051ea94b035097720d1d9ff18da86
=> => extracting sha256:f4d2ddb8b5ad45a0a94834d8865d45a83897e38f129ff5d7d3ce61c2c477f9d8
=> => extracting sha256:a67e1278418011fe3e3a3ac21061850263753dcab6c6fb4e793271526c6b8666f
=> [ 2/10] COPY . /vim-emu
=> [ 3/10] WORKDIR /vim-emu/ansible
=> [ 4/10] RUN ansible-playbook -i "localhost," -c local --skip-tags "notindocker" install.yml
=> [ 5/10] WORKDIR /vim-emu
=> [ 6/10] RUN python3 setup.py develop
=> [ 7/10] RUN pip3 install "tinypc==1.0.3"
=> [ 8/10] RUN pip3 install wheel --upgrade
=> [ 9/10] RUN mv /usr/bin/python /usr/bin/python2 && ln -s /usr/bin/python3 /usr/bin/python &
=> => exporting to image
=> => exporting layers
```

Step 3:

Run the interactive container by using the command,

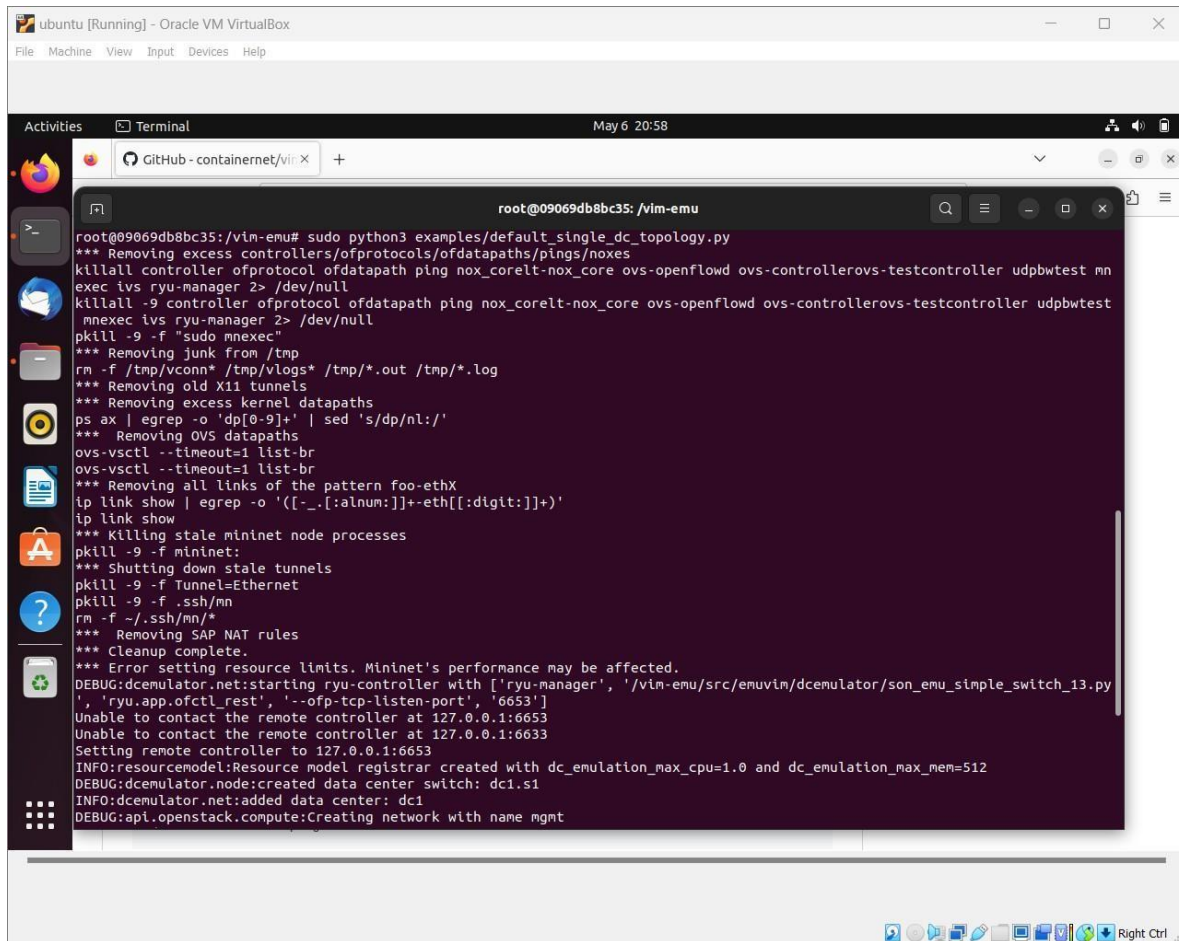
```
docker run --name vim-emu -it --rm --privileged --pid='host' -v  
/var/run/docker.sock:/var/run/docker.sock vim-emu-img /bin/bash
```



Step 4:

Start the emulator with a simple topology (terminal 1).

```
# start the emulation platform with a single NFV data center  
sudo python3 examples/default_single_dc_topology.py
```



```
ubuntu [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

Activities Terminal May 6 21:27

root@09069db8bc35: /vim-emu

root@09069db8bc35: /vim-emu x ubuntu@ubuntu

*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Removing SAP NAT rules
*** Cleanup complete.
*** Error setting resource limits. Mininet's performance may be affected.
DEBUG:dcemulator.net:starting ryu-controller with ['ryu-manager', '/vim-emu/src/emuvim/dcemulator/.app.ofctl_rest', '--ofp-tcp-listen-port', '6653']
Unable to contact the remote controller at 127.0.0.1:6653
Unable to contact the remote controller at 127.0.0.1:6633
Setting remote controller to 127.0.0.1:6653
INFO:resourcemodel:Resource model registrar created with dc_emulation_max_cpu=1.0 and dc_emulation_
DEBUG:dcemulator.node:created data center switch: dc1.s1
INFO:dcemulator.net:added data center: dc1
DEBUG:api.openstack.compute:Creating network with name mgmt
name_or_id: mgmt
WARNING:api.openstack.compute:Could not find net 'mgmt' in dict_keys([]) or []
DEBUG:api.openstack.compute:Creating network with name mgmtnet
name_or_id: mgmtnet
WARNING:api.openstack.compute:Could not find net 'mgmtnet' in dict_keys(['91b972f7-27ba-4bb9-8ebb-9
INFO:api.openstack.base:Starting KeystoneDummyApi endpoint @ http://0.0.0.0:6001
INFO:api.openstack.base:Starting NeutronDummyApi endpoint @ http://0.0.0.0:10697
INFO:api.openstack.base:Starting NovaDummyApi endpoint @ http://0.0.0.0:9775
INFO:api.openstack.base:Starting HeatDummyApi endpoint @ http://0.0.0.0:9005
INFO:api.openstack.base:Starting GlanceDummyApi endpoint @ http://0.0.0.0:10243
DEBUG:dcemulator.net:addLink: n1=root intf1=root-eth0 -- n2=fs1 intf2=fs1-eth1
*** Configuring hosts
root
*** Starting controller
c0
*** Starting 2 switches
dc1.s1 fs1 ...
*** Starting CLI:
containernet>
```

a) Open second terminal for nested Docker deployment using the following command,

```
sudo docker exec -it 09069db8bc35 bash
```

```
Execute a command in a running container
ubuntu@ubuntu:~/vim-emu$ sudo docker exec -it 09069db8bc35 bash
root@09069db8bc35:/vim-emu#
```

b) Start two simple VNFs using the following commands(in the second terminal).

```
vim-emu compute start -d dc1 -n vnf1
vim-emu compute start -d dc1 -n vnf2
vim-emu compute list
```


- c) Check the connectivity between the two VNFs using following commands (in first terminal)

```
containernet> vnf1 ifconfig  
containernet> vnf1 ping -c2 vnf2
```

Result:

Thus the simple end-to-end network service with two VNFs using vim-emu <https://github.com/containernet/vim-emu> was created successfully.

Ex. No: 5
DATE:

Install OSM and onboard and orchestrate network service

AIM:

To install OSM and onboard and orchestrate network service.

ALGORITHM:

Step 1: Goto the website <https://osm.etsi.org>.

Step 2: Click on Try OSM. And then click How to install OSM.

Step 3: Download and install OSM.

Step 4: Install a standalone Kubernetes on a single host, and OSM on top of it.

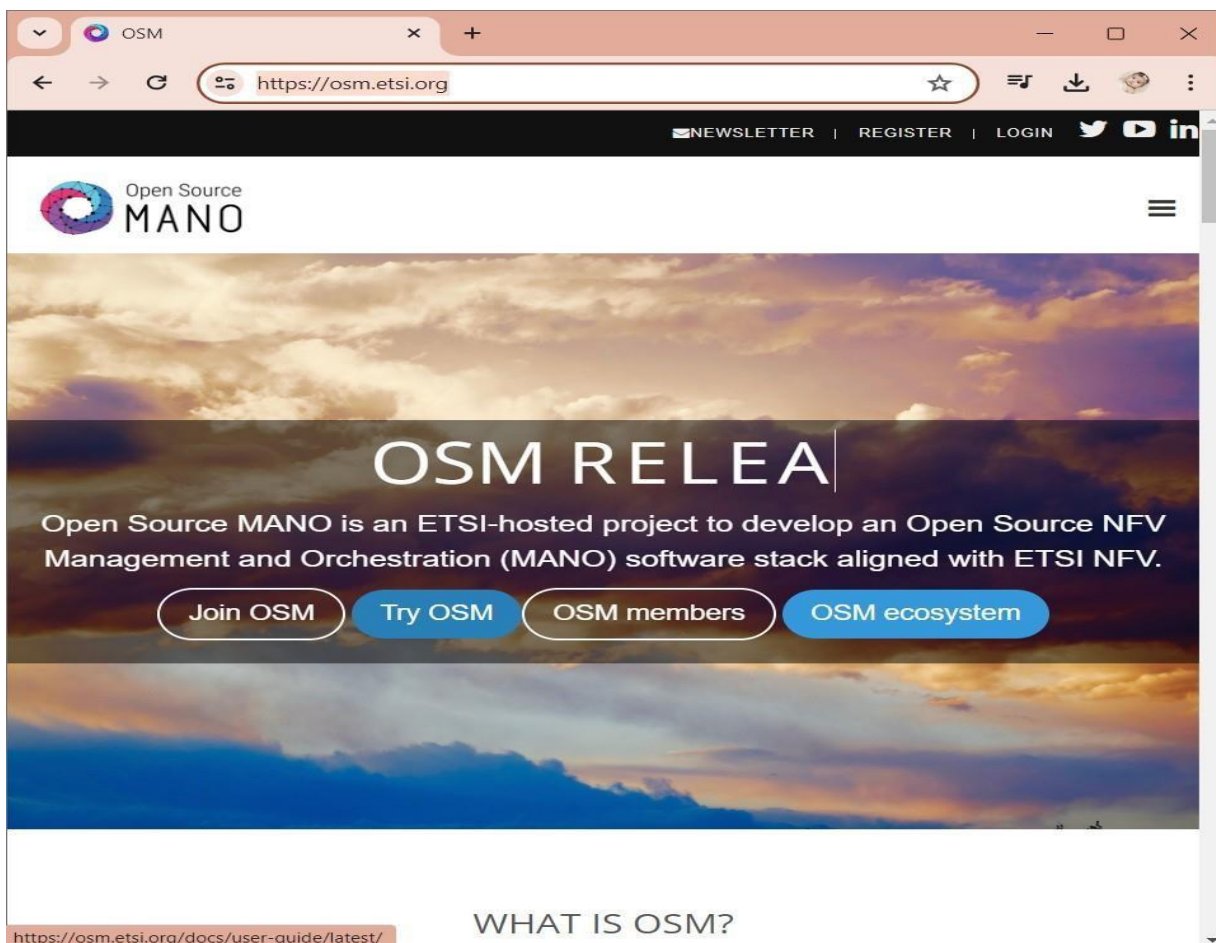
Step 5: Install Juju and LXD, which is required to deploy VNFs that use Execution Environments based on Juju charms.

Step 6: Check whether the OSM installed properly or not.

PROCEDURE:

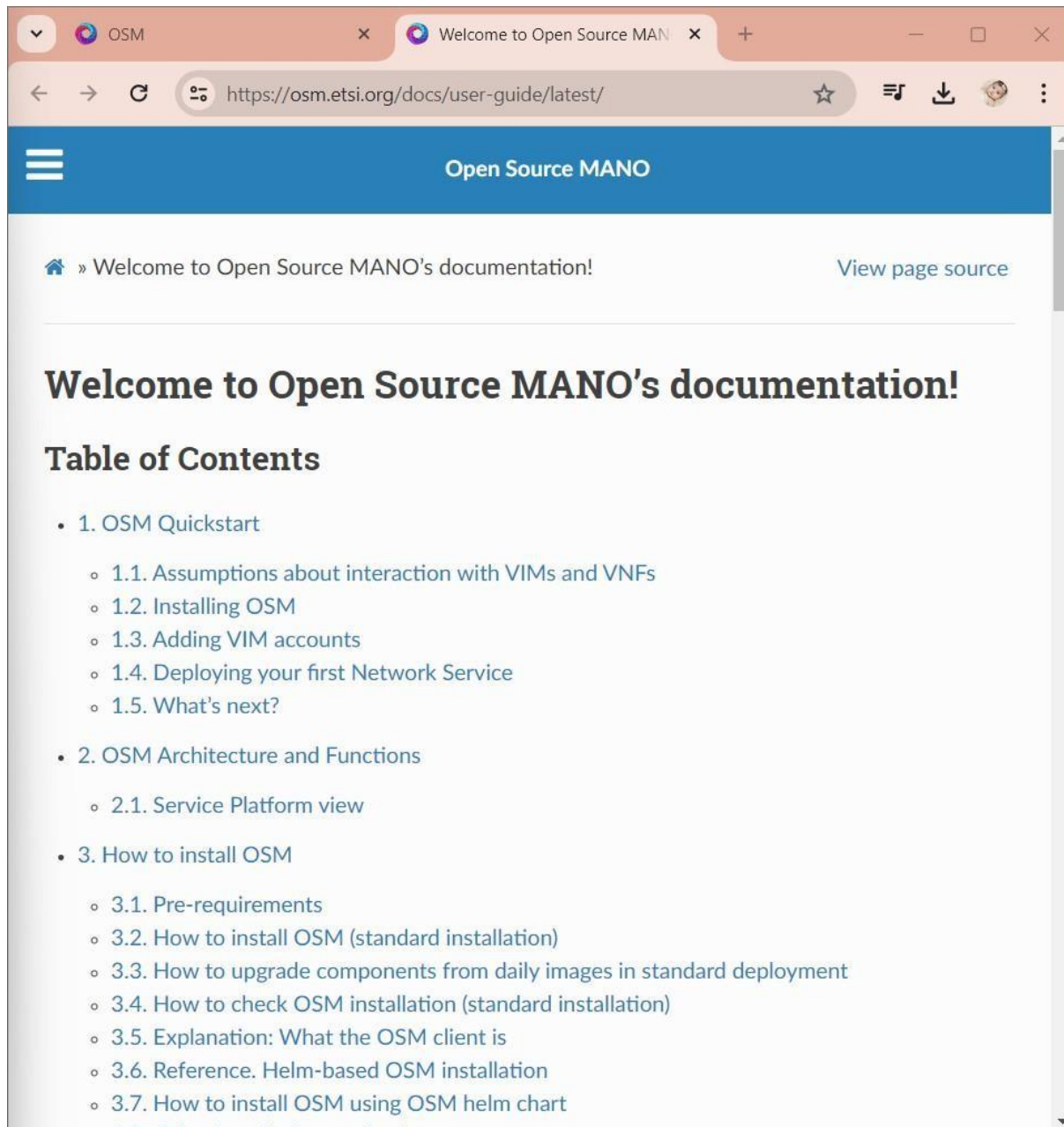
Step 1:

Goto the website <https://osm.etsi.org>.



Step

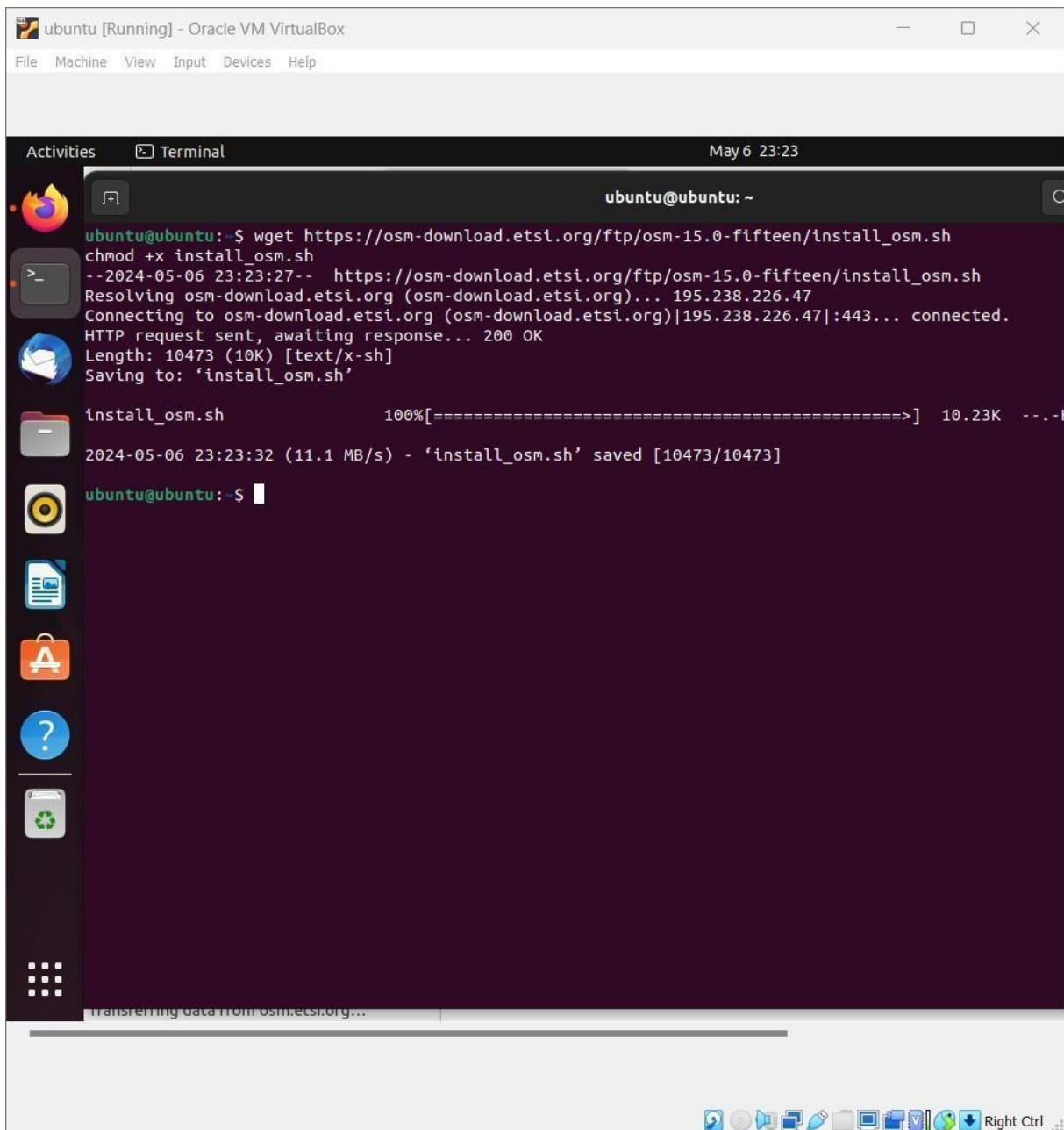
2: Click on Try OSM. And then click Installing OSM.



Step 3:

Download and install OSM using the following command,

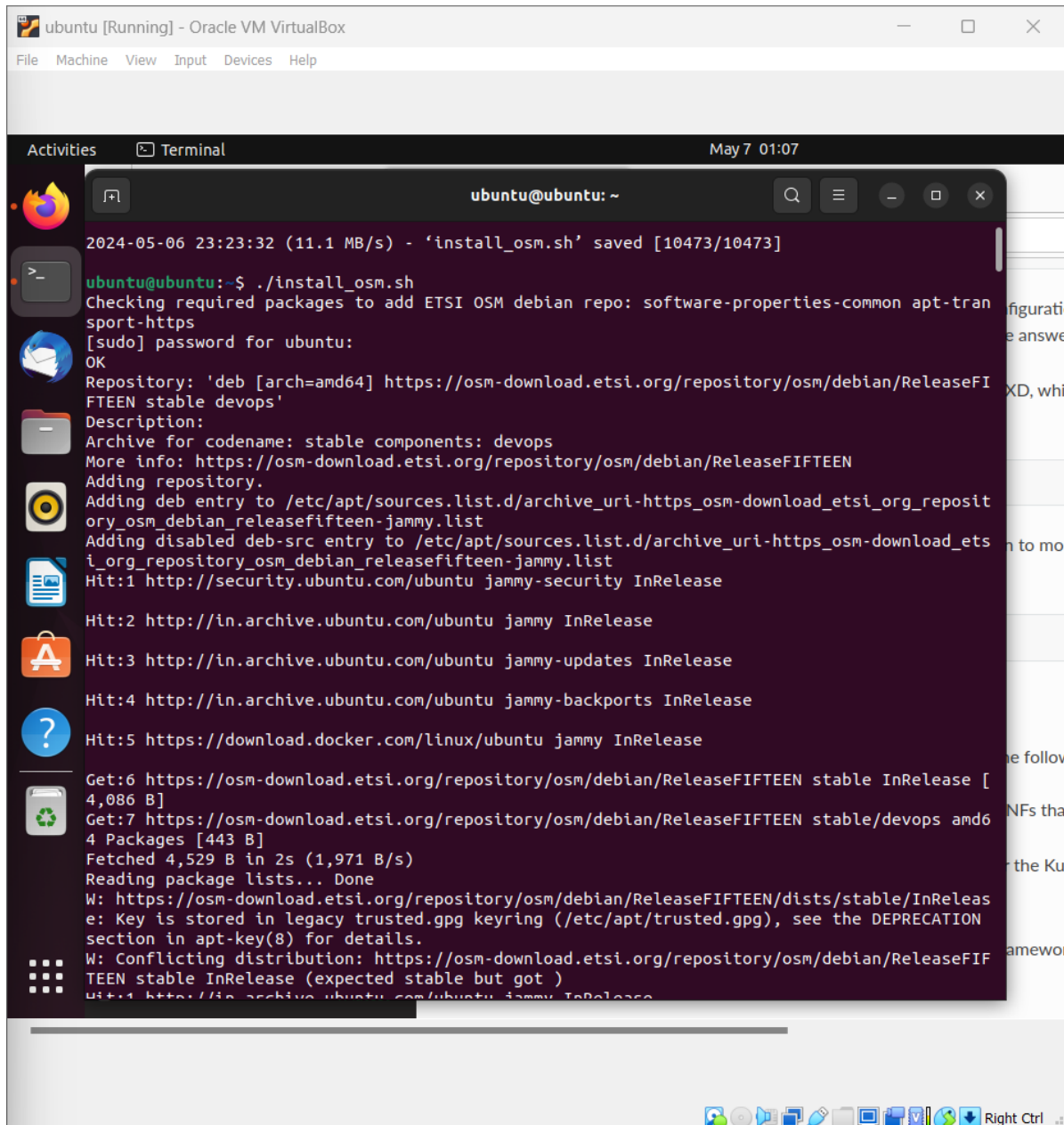
```
wget https://osm-download.etsi.org/ftp/osm-15.0-fifteen/install_osm.sh chmod +x install_osm.sh
```

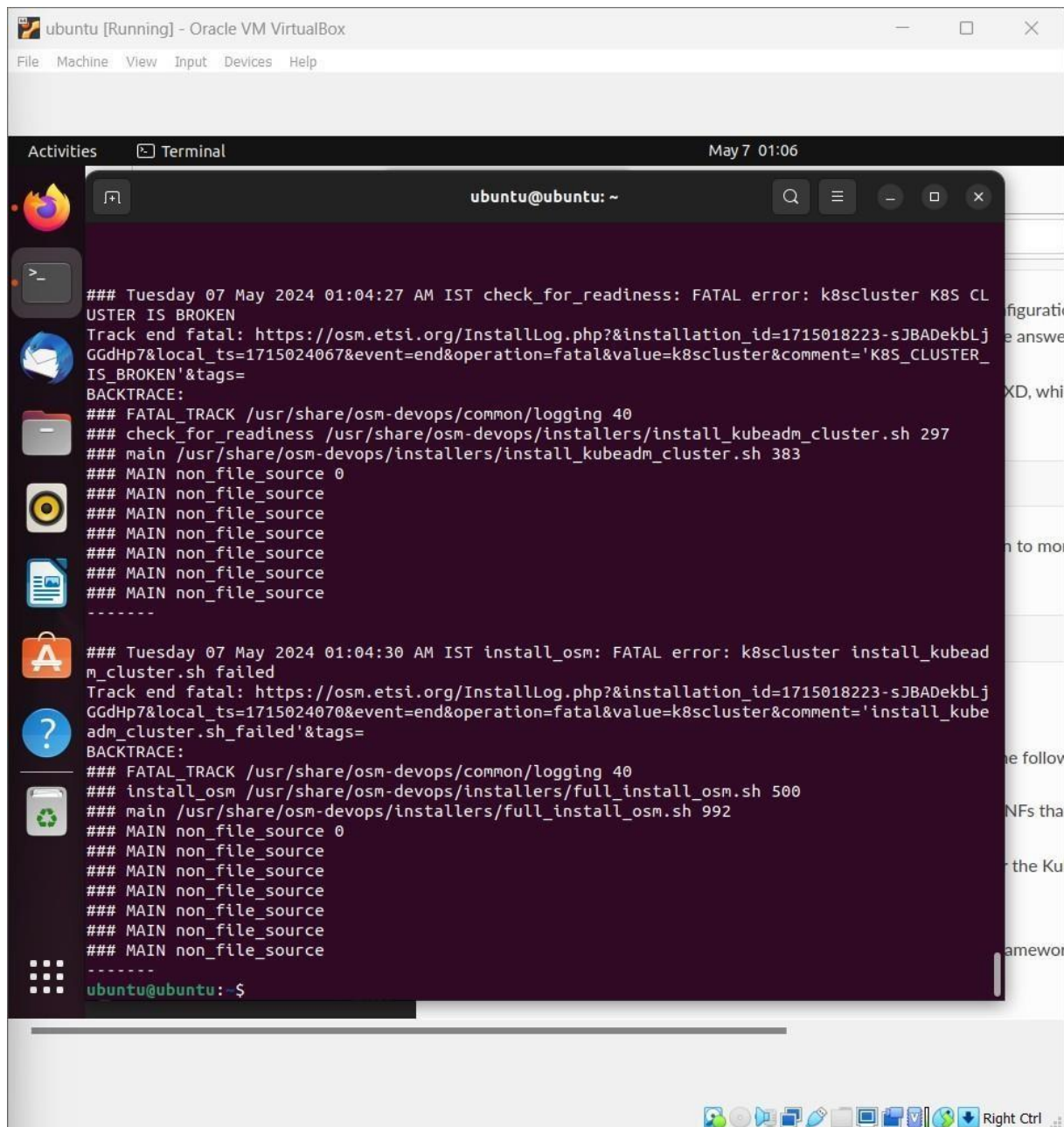


Step 4:

Install a standalone Kubernetes on a single host, and OSM on top of it.

`./install_osm.sh`





The screenshot shows a terminal window titled 'ubuntu [Running] - Oracle VM VirtualBox'. The terminal output displays two fatal error messages. The first message, timestamped 'Tuesday 07 May 2024 01:04:27 AM IST', reports a 'FATAL error: k8scluster K8S CLUSTER IS BROKEN' and includes a backtrace showing the failure in 'install_kubeadm_cluster.sh' at line 297. The second message, timestamped 'Tuesday 07 May 2024 01:04:30 AM IST', reports a 'FATAL error: k8scluster install_kubeadm_cluster.sh failed' and includes a backtrace showing the failure in 'full_install_osm.sh' at line 500. Both messages include a 'Track end fatal' URL and a 'BACKTRACE' section listing various file sources.

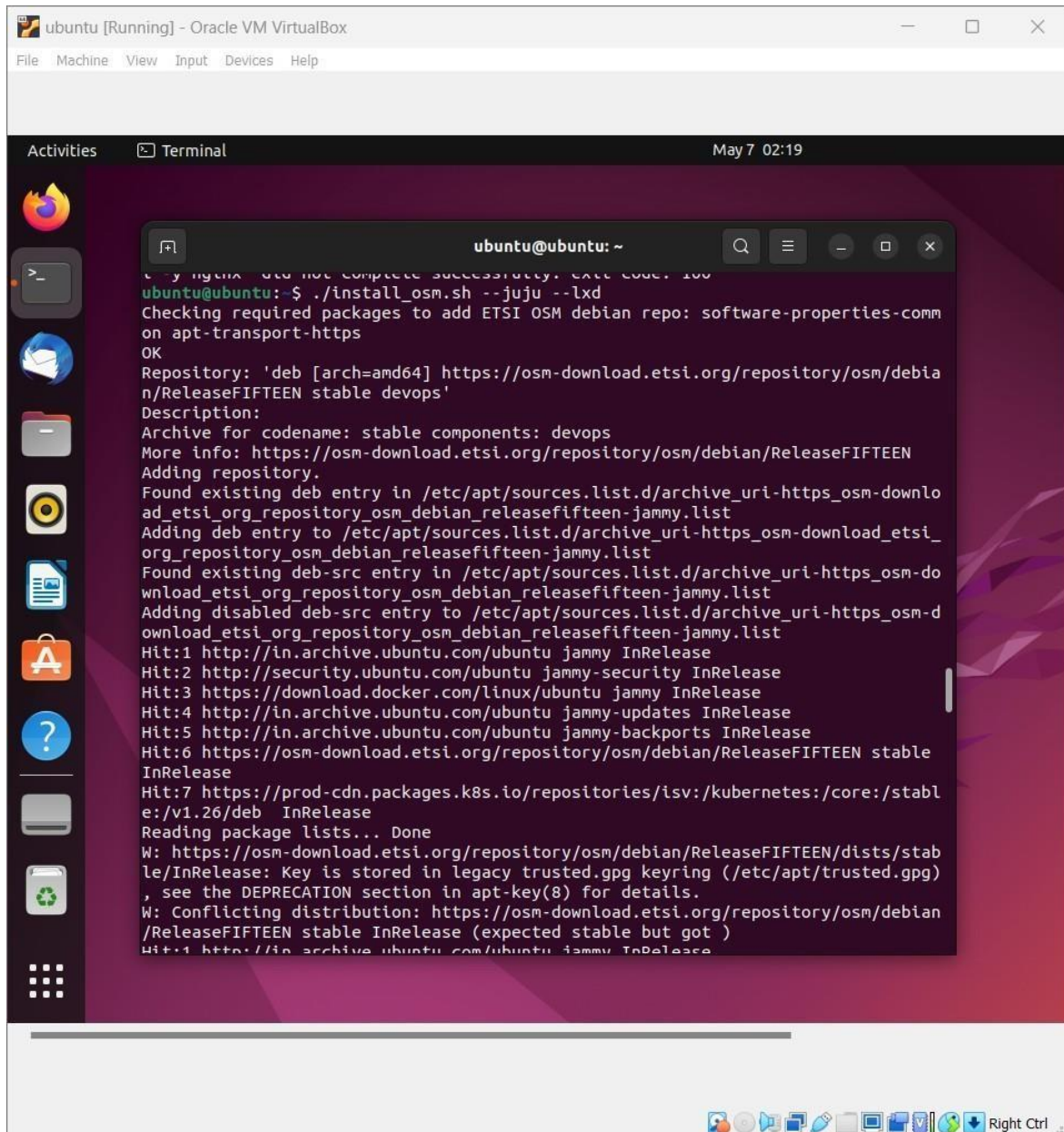
```
### Tuesday 07 May 2024 01:04:27 AM IST check_for_readiness: FATAL error: k8scluster K8S CLUSTER IS BROKEN
Track end fatal: https://osm.etsi.org/InstallLog.php?&installation_id=1715018223-sJBADekbLjGGdHp7&local_ts=1715024067&event=end&operation=fatal&value=k8scluster&comment='K8S_CLUSTER_IS_BROKEN'&tags=
BACKTRACE:
### FATAL_TRACK /usr/share/osm-devops/common/logging 40
### check_for_readiness /usr/share/osm-devops/installers/install_kubeadm_cluster.sh 297
### main /usr/share/osm-devops/installers/install_kubeadm_cluster.sh 383
### MAIN non_file_source 0
### MAIN non_file_source
### MAIN non_file_source
### MAIN non_file_source
### MAIN non_file_source
### MAIN non_file_source
### MAIN non_file_source
-----

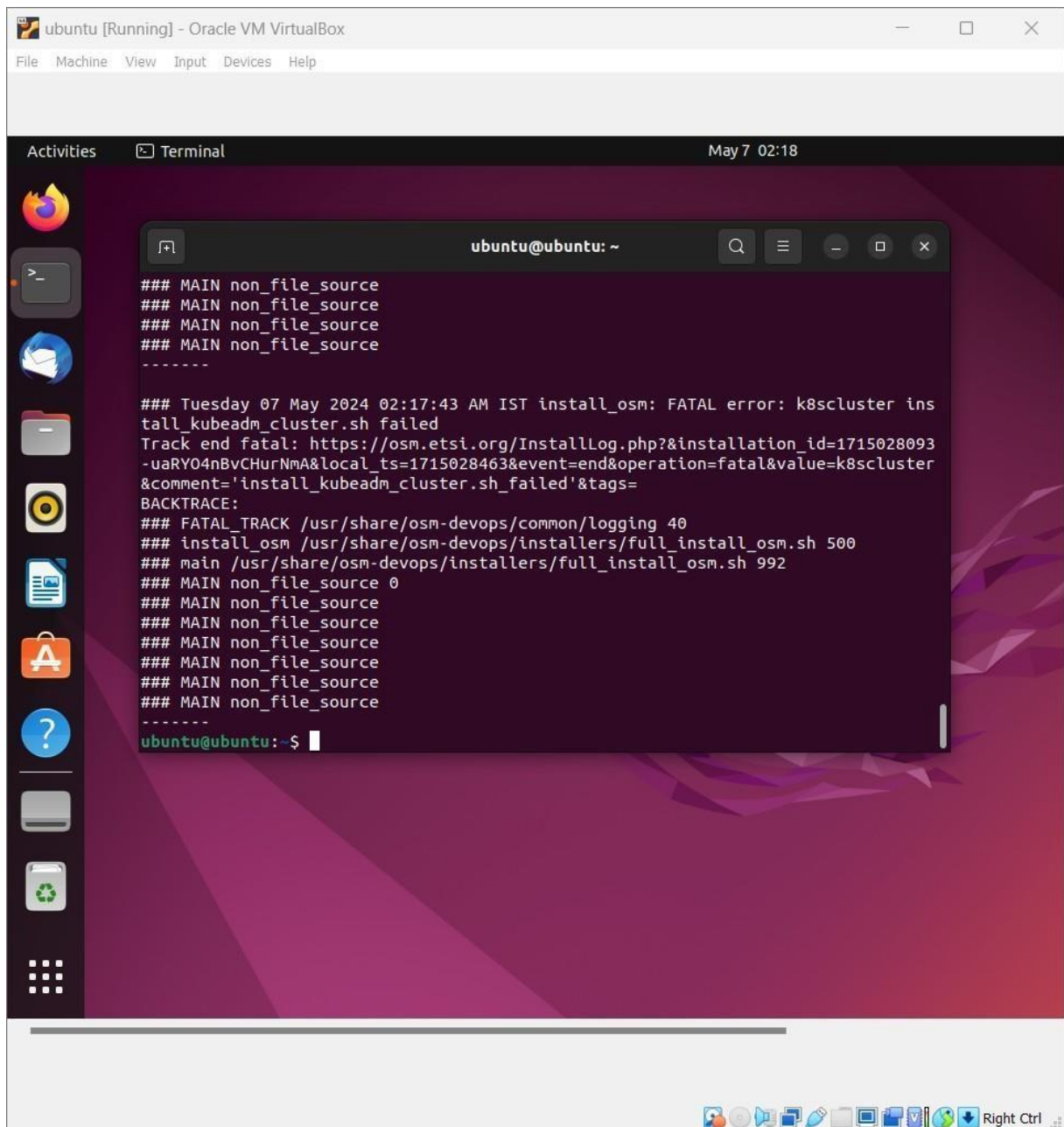
### Tuesday 07 May 2024 01:04:30 AM IST install_osm: FATAL error: k8scluster install_kubeadm_cluster.sh failed
Track end fatal: https://osm.etsi.org/InstallLog.php?&installation_id=1715018223-sJBADekbLjGGdHp7&local_ts=1715024070&event=end&operation=fatal&value=k8scluster&comment='install_kubeadm_cluster.sh_failed'&tags=
BACKTRACE:
### FATAL_TRACK /usr/share/osm-devops/common/logging 40
### install_osm /usr/share/osm-devops/installers/full_install_osm.sh 500
### main /usr/share/osm-devops/installers/full_install_osm.sh 992
### MAIN non_file_source 0
### MAIN non_file_source
### MAIN non_file_source
### MAIN non_file_source
### MAIN non_file_source
### MAIN non_file_source
### MAIN non_file_source
-----
ubuntu@ubuntu: ~$
```

Step 5:

Install juji and LXD, which is required to deploy VNFs that use executive environment based on juji charms

```
./install_osm.sh --juju --lxd
```

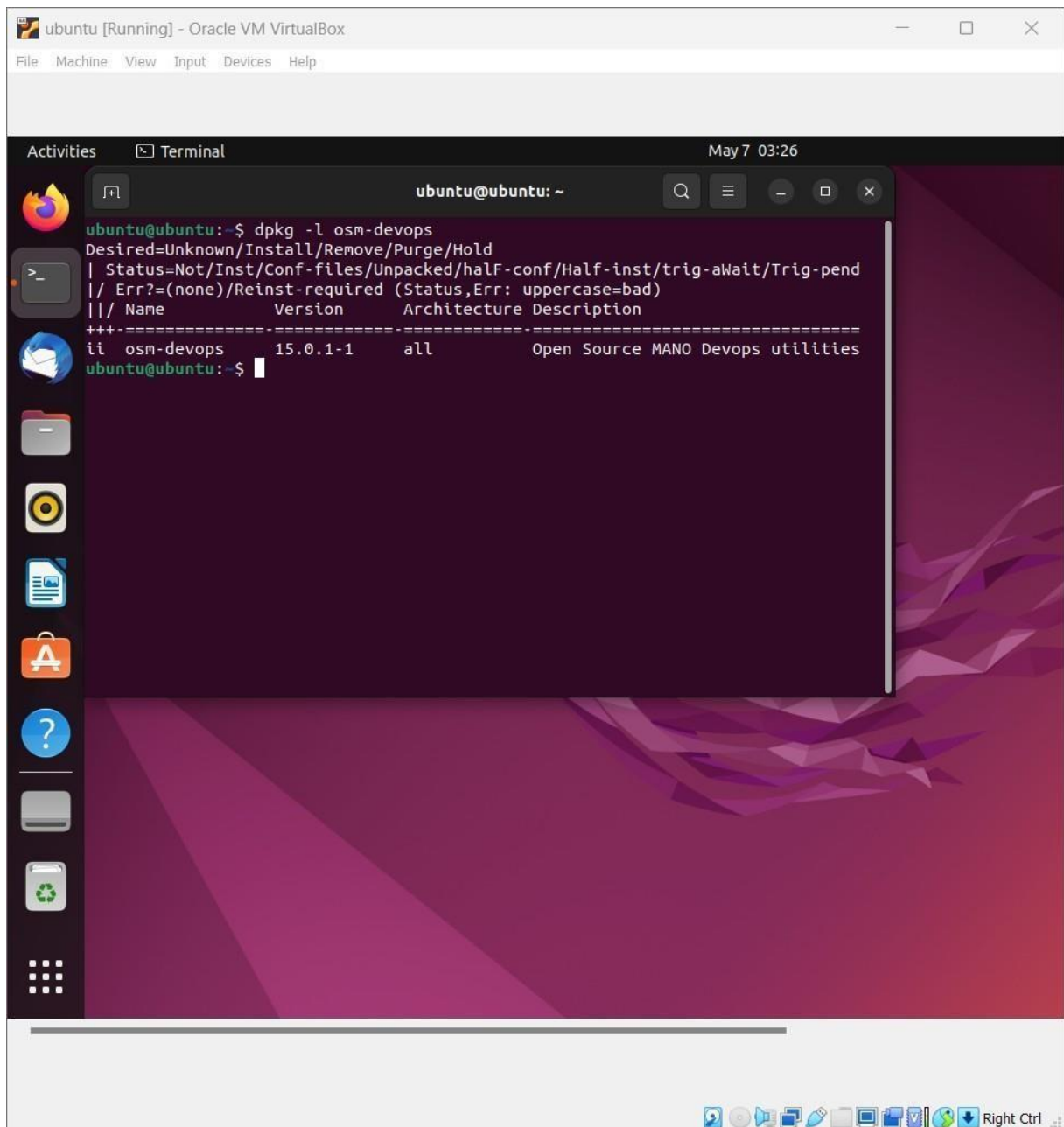




Step 6:

Check whether the OSM installed properly or not using the following command,

`dpkg -l osm-devops`



Result:

Thus the OSM was installed successfully and network services are onboarded successfully.